

Dataset

October 3, 2026

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

LOAN APPLICATION DATA ANALYSIS REPORT

Exploratory Data Analysis (EDA) Using Python, Pandas & Matplotlib

0.0.1 Prepared By

1 Sylvester Ojwang

1.0.1 Registration Number

1.1 Institution

1.1.1 Jomo Kenyatta University of Agriculture and Technology

1.1.2 School of Computing and Information Technology

Course	Date
Data Science with Python	21 June 2026

Lecturer	Project
Dr. _____Michael Locky_____	Loan Approval Prediction Analysis

1.2 Technologies Used

2 PROJECT WORKFLOW

CSV DATASET

DATA IMPORT (Pandas)

DATA EXPLORATION

DATA CLEANING

EXPLORATORY DATA ANALYSIS

DATA VISUALIZATION

BUSINESS INSIGHTS

RECOMMENDATIONS

FINAL REPORT

3 Executive Summary

This report presents a comprehensive exploratory data analysis (EDA) of a bank loan application dataset. The analysis focuses on understanding applicant characteristics, identifying trends influencing loan approval decisions, handling missing values, detecting outliers, and producing actionable business insights through statistical analysis and data visualization.

Version 1.0

© 2026 Sylvester Ojwang

```
[2]: df =pd.read_csv(r"C:\Users\User\Downloads\bankloan.csv")
df.head()
```

```
[2]:   Loan_ID Gender Married Dependents Education Self_Employed \
0  LP001002  Male      No         0.0   Graduate             No
1  LP001003  Male     Yes         1.0   Graduate             No
```

2	LP001005	Male	Yes	0.0	Graduate	Yes
3	LP001006	Male	Yes	0.0	Not Graduate	No
4	LP001008	Male	No	0.0	Graduate	No

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	NaN	360.0	
1	4583	1508.0	128.0	360.0	
2	3000	0.0	66.0	360.0	
3	2583	2358.0	120.0	360.0	
4	6000	0.0	141.0	360.0	

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y
1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

```
[3]: drop=df.drop("Loan_ID",axis=1)
drop
```

```
[3]:
```

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	\
0	Male	No	0.0	Graduate	No	5849	
1	Male	Yes	1.0	Graduate	No	4583	
2	Male	Yes	0.0	Graduate	Yes	3000	
3	Male	Yes	0.0	Not Graduate	No	2583	
4	Male	No	0.0	Graduate	No	6000	
..	...	...	...	...	...	...	
609	Female	No	0.0	Graduate	No	2900	
610	Male	Yes	3.0	Graduate	No	4106	
611	Male	Yes	1.0	Graduate	No	8072	
612	Male	Yes	2.0	Graduate	No	7583	
613	Female	No	0.0	Graduate	Yes	4583	

	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	\
0	0.0	NaN	360.0	1.0	
1	1508.0	128.0	360.0	1.0	
2	0.0	66.0	360.0	1.0	
3	2358.0	120.0	360.0	1.0	
4	0.0	141.0	360.0	1.0	
..	...	...	...	...	
609	0.0	71.0	360.0	1.0	
610	0.0	40.0	180.0	1.0	
611	240.0	253.0	360.0	1.0	
612	0.0	187.0	360.0	1.0	
613	0.0	133.0	360.0	0.0	

	Property_Area	Loan_Status
0	Urban	Y
1	Rural	N
2	Urban	Y
3	Urban	Y
4	Urban	Y
..	...	...
609	Rural	Y
610	Rural	Y
611	Urban	Y
612	Urban	Y
613	Semiurban	N

[614 rows x 12 columns]

```
[4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                614 non-null   object
1   Gender                 601 non-null   object
2   Married                611 non-null   object
3   Dependents             599 non-null   float64
4   Education              614 non-null   object
5   Self_Employed          582 non-null   object
6   ApplicantIncome        614 non-null   int64
7   CoapplicantIncome      614 non-null   float64
8   LoanAmount             592 non-null   float64
9   Loan_Amount_Term       600 non-null   float64
10  Credit_History         564 non-null   float64
11  Property_Area          614 non-null   object
12  Loan_Status            614 non-null   object
dtypes: float64(5), int64(1), object(7)
memory usage: 62.5+ KB
```

```
[5]: df.tail()
```

```
[5]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
609	LP002978	Female	No	0.0	Graduate	No	
610	LP002979	Male	Yes	3.0	Graduate	No	
611	LP002983	Male	Yes	1.0	Graduate	No	
612	LP002984	Male	Yes	2.0	Graduate	No	
613	LP002990	Female	No	0.0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
--	-----------------	-------------------	------------	------------------	---

609	2900	0.0	71.0	360.0
610	4106	0.0	40.0	180.0
611	8072	240.0	253.0	360.0
612	7583	0.0	187.0	360.0
613	4583	0.0	133.0	360.0

	Credit_History	Property_Area	Loan_Status
609	1.0	Rural	Y
610	1.0	Rural	Y
611	1.0	Urban	Y
612	1.0	Urban	Y
613	0.0	Semiurban	N

```
[6]: df.dtypes
```

```
[6]: Loan_ID      object
Gender          object
Married         object
Dependents      float64
Education       object
Self_Employed   object
ApplicantIncome int64
CoapplicantIncome float64
LoanAmount      float64
Loan_Amount_Term float64
Credit_History  float64
Property_Area   object
Loan_Status     object
dtype: object
```

```
[7]: df.describe()
```

```
[7]:
```

	Dependents	ApplicantIncome	CoapplicantIncome	LoanAmount	\
count	599.000000	614.000000	614.000000	592.000000	
mean	0.762938	5403.459283	1621.245798	146.412162	
std	1.015216	6109.041673	2926.248369	85.587325	
min	0.000000	150.000000	0.000000	9.000000	
25%	0.000000	2877.500000	0.000000	100.000000	
50%	0.000000	3812.500000	1188.500000	128.000000	
75%	2.000000	5795.000000	2297.250000	168.000000	
max	3.000000	81000.000000	41667.000000	700.000000	

	Loan_Amount_Term	Credit_History
count	600.00000	564.000000
mean	342.00000	0.842199
std	65.12041	0.364878
min	12.00000	0.000000

25%	360.00000	1.000000
50%	360.00000	1.000000
75%	360.00000	1.000000
max	480.00000	1.000000

```
[8]: df.shape
```

```
[8]: (614, 13)
```

```
[9]: df.tail()
```

```
[9]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
609	LP002978	Female	No	0.0	Graduate	No	
610	LP002979	Male	Yes	3.0	Graduate	No	
611	LP002983	Male	Yes	1.0	Graduate	No	
612	LP002984	Male	Yes	2.0	Graduate	No	
613	LP002990	Female	No	0.0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
609	2900	0.0	71.0	360.0	
610	4106	0.0	40.0	180.0	
611	8072	240.0	253.0	360.0	
612	7583	0.0	187.0	360.0	
613	4583	0.0	133.0	360.0	

	Credit_History	Property_Area	Loan_Status
609	1.0	Rural	Y
610	1.0	Rural	Y
611	1.0	Urban	Y
612	1.0	Urban	Y
613	0.0	Semiurban	N

```
[10]: df["Gender"].describe()
```

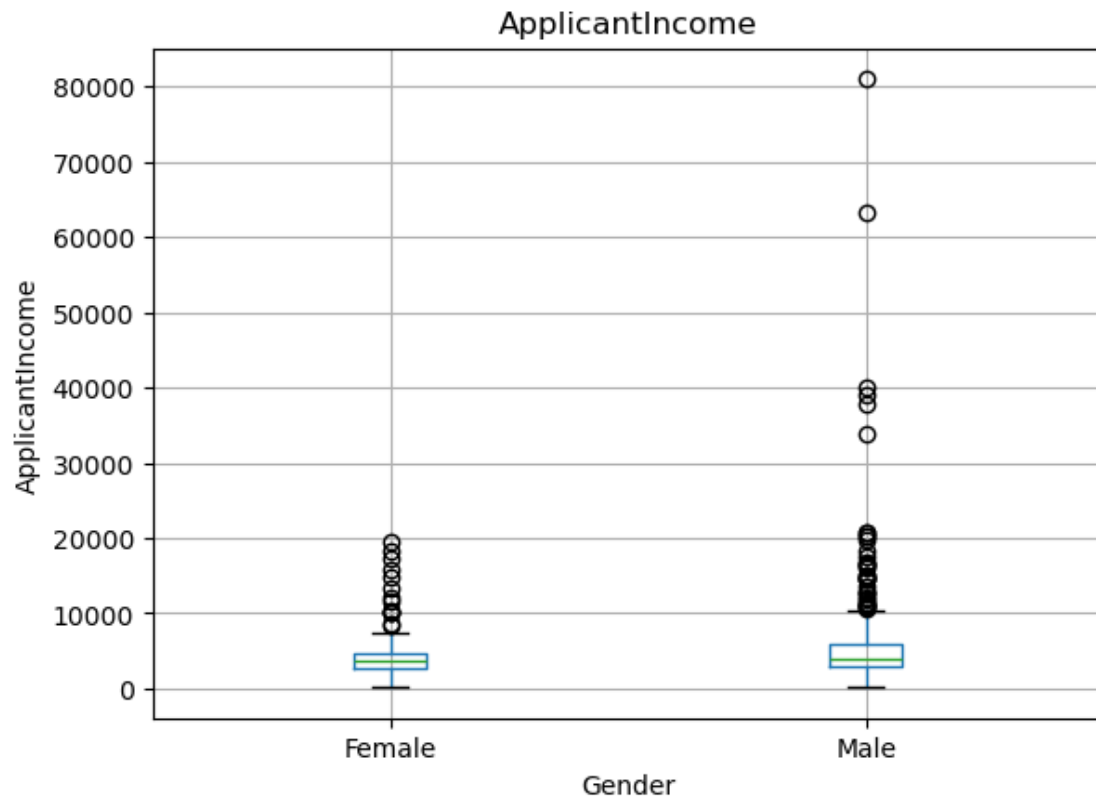
```
[10]: count      601
unique         2
top      Male
freq         489
Name: Gender, dtype: object
```

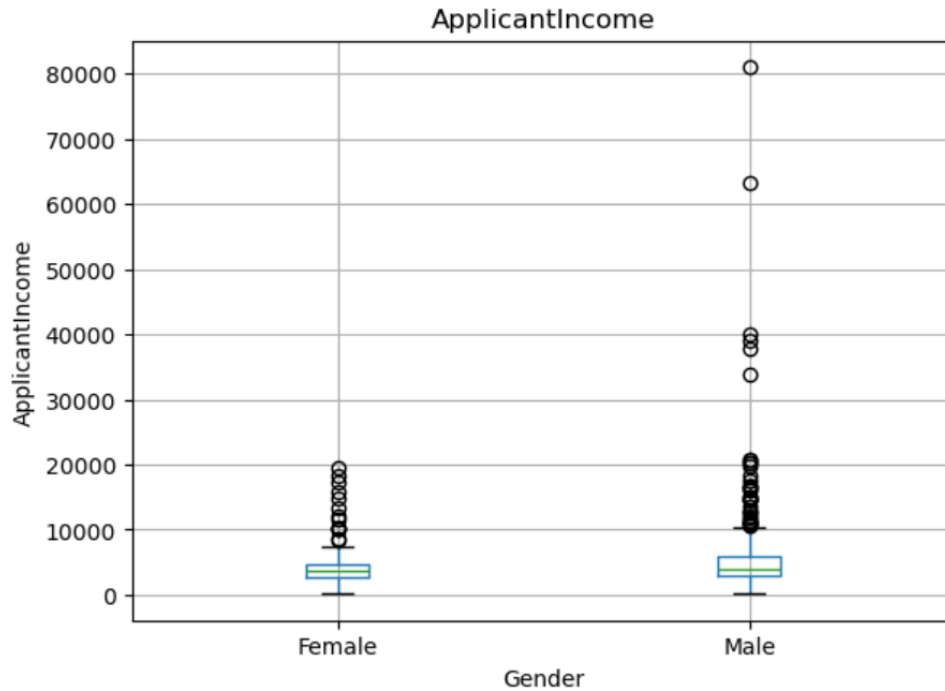
```
[11]: df["ApplicantIncome"].describe()
```

```
[11]: count      614.000000
mean      5403.459283
std      6109.041673
min      150.000000
25%      2877.500000
50%      3812.500000
```

```
75%      5795.000000
max      81000.000000
Name: ApplicantIncome, dtype: float64
```

```
[12]: df.boxplot(column="ApplicantIncome", by="Gender")
plt.suptitle("")
plt.xlabel("Gender")
plt.ylabel("ApplicantIncome")
plt.show()
```





```
[13]: df.shape
```

```
[13]: (614, 13)
```

```
[14]: df.isnull().sum()
```

```
[14]: Loan_ID      0
      Gender      13
      Married     3
      Dependents  15
      Education   0
      Self_Employed  32
      ApplicantIncome  0
      CoapplicantIncome  0
      LoanAmount    22
      Loan_Amount_Term  14
      Credit_History  50
      Property_Area   0
      Loan_Status    0
      dtype: int64
```

```
[15]: df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])

      df["Married"] = df["Married"].fillna(df["Married"].mode()[0])

      df["Dependents"] = df["Dependents"].fillna(df["Dependents"].mode()[0])
```

```
df["Self_Employed"] = df["Self_Employed"].fillna(df["Self_Employed"].mode()[0])
```

```
[16]: df["LoanAmount"] = df["LoanAmount"].fillna(df["LoanAmount"].mean())

df["Loan_Amount_Term"] = df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"].
↳mean())

df["Credit_History"] = df["Credit_History"].fillna(df["Credit_History"].
↳mode()[0])
```

```
[17]: df.isnull().sum()
```

```
[17]: Loan_ID      0
Gender          0
Married         0
Dependents      0
Education       0
Self_Employed  0
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount      0
Loan_Amount_Term 0
Credit_History  0
Property_Area   0
Loan_Status     0
dtype: int64
```

```
[18]: df.iloc[:, 0:8]
```

```
[18]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	Male	No	0.0	Graduate	No	
1	LP001003	Male	Yes	1.0	Graduate	No	
2	LP001005	Male	Yes	0.0	Graduate	Yes	
3	LP001006	Male	Yes	0.0	Not Graduate	No	
4	LP001008	Male	No	0.0	Graduate	No	
..	...	...	...	...	...	...	
609	LP002978	Female	No	0.0	Graduate	No	
610	LP002979	Male	Yes	3.0	Graduate	No	
611	LP002983	Male	Yes	1.0	Graduate	No	
612	LP002984	Male	Yes	2.0	Graduate	No	
613	LP002990	Female	No	0.0	Graduate	Yes	
	ApplicantIncome	CoapplicantIncome					
0	5849	0.0					
1	4583	1508.0					
2	3000	0.0					

3	2583	2358.0
4	6000	0.0
..	...	...
609	2900	0.0
610	4106	0.0
611	8072	240.0
612	7583	0.0
613	4583	0.0

[614 rows x 8 columns]

```
[19]: df.pivot_table(index = "Education", values = [
    ↪ "ApplicantIncome", "LoanAmount", "Loan_Amount_Term"], aggfunc=
    ↪ ["sum", "count", "mean", "max", "min"])
```

```
[19]:
```

	sum		count \	
	ApplicantIncome	LoanAmount	Loan_Amount_Term	ApplicantIncome
Education				
Graduate	2811568	73834.182432	165420.0	480
Not Graduate	506156	16062.885135	44568.0	134

	mean \			
	LoanAmount	Loan_Amount_Term	ApplicantIncome	LoanAmount
Education				
Graduate	480	480	5857.433333	153.821213
Not Graduate	134	134	3777.283582	119.872277

	max \			
	Loan_Amount_Term	ApplicantIncome	LoanAmount	Loan_Amount_Term
Education				
Graduate	344.625000	81000	700.0	480.0
Not Graduate	332.597015	18165	279.0	480.0

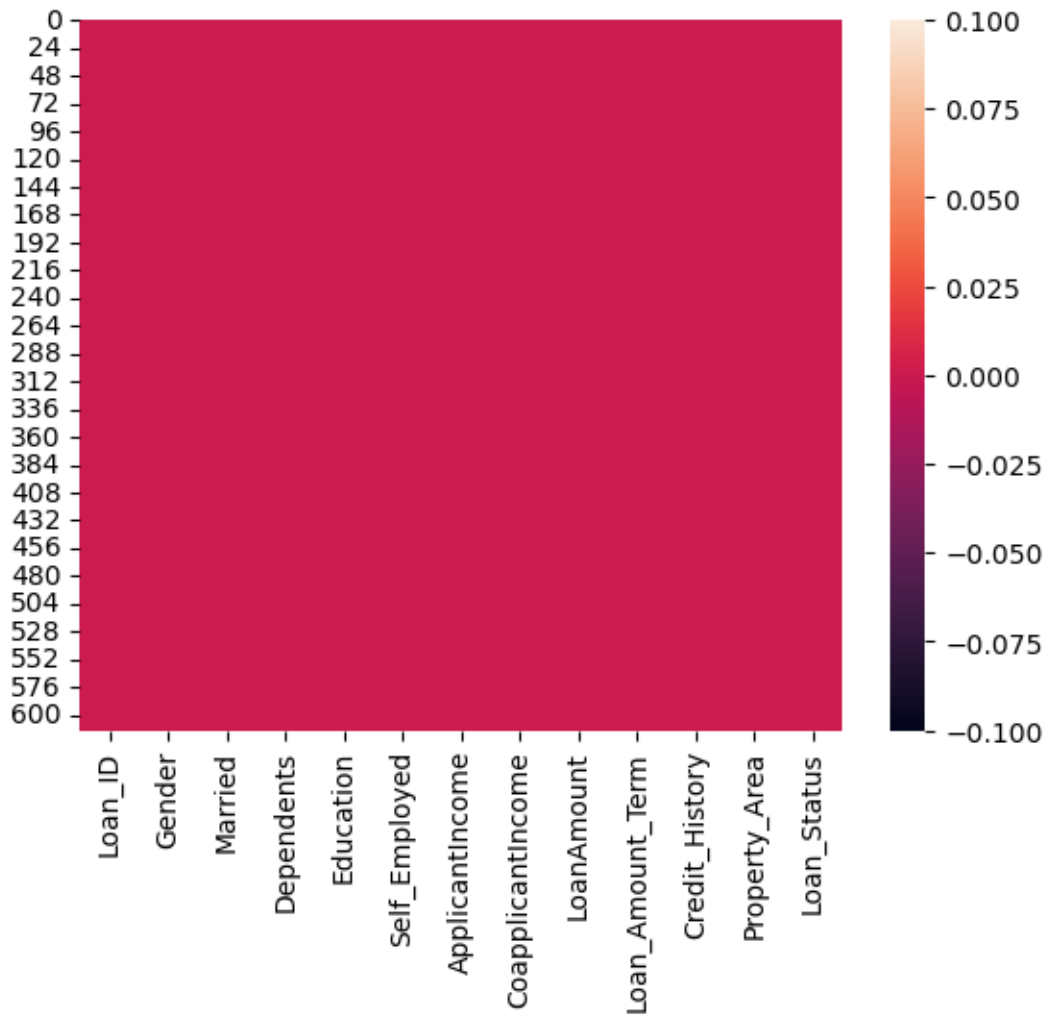
	min		
	ApplicantIncome	LoanAmount	Loan_Amount_Term
Education			
Graduate	150	9.0	12.0
Not Graduate	210	25.0	36.0

```
[20]: df.drop_duplicates(inplace = True)
df[df.duplicated()]
```

```
[20]: Empty DataFrame
Columns: [Loan_ID, Gender, Married, Dependents, Education, Self_Employed,
ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term,
Credit_History, Property_Area, Loan_Status]
Index: []
```

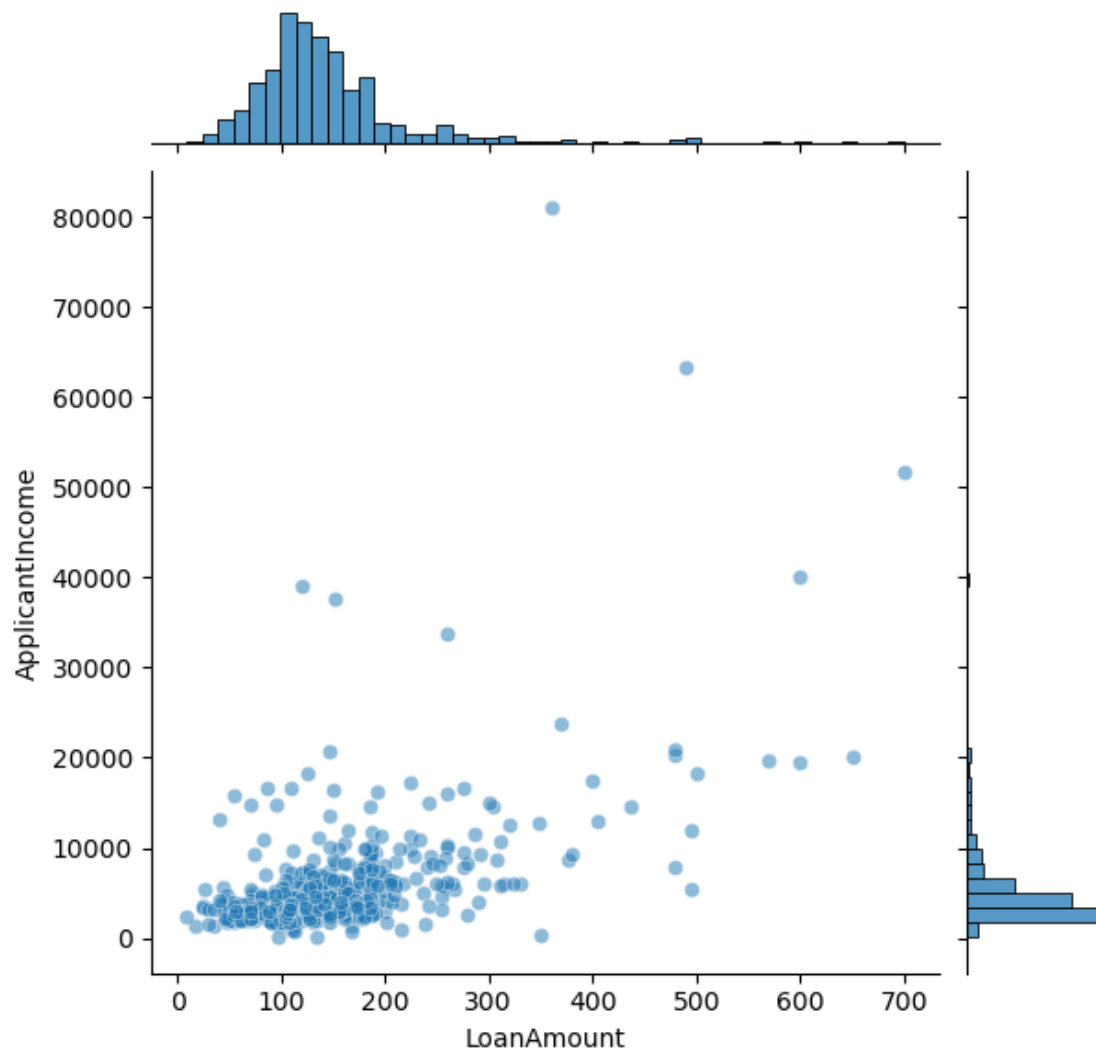
```
[21]: sns.heatmap(df.isnull())
```

```
[21]: <Axes: >
```



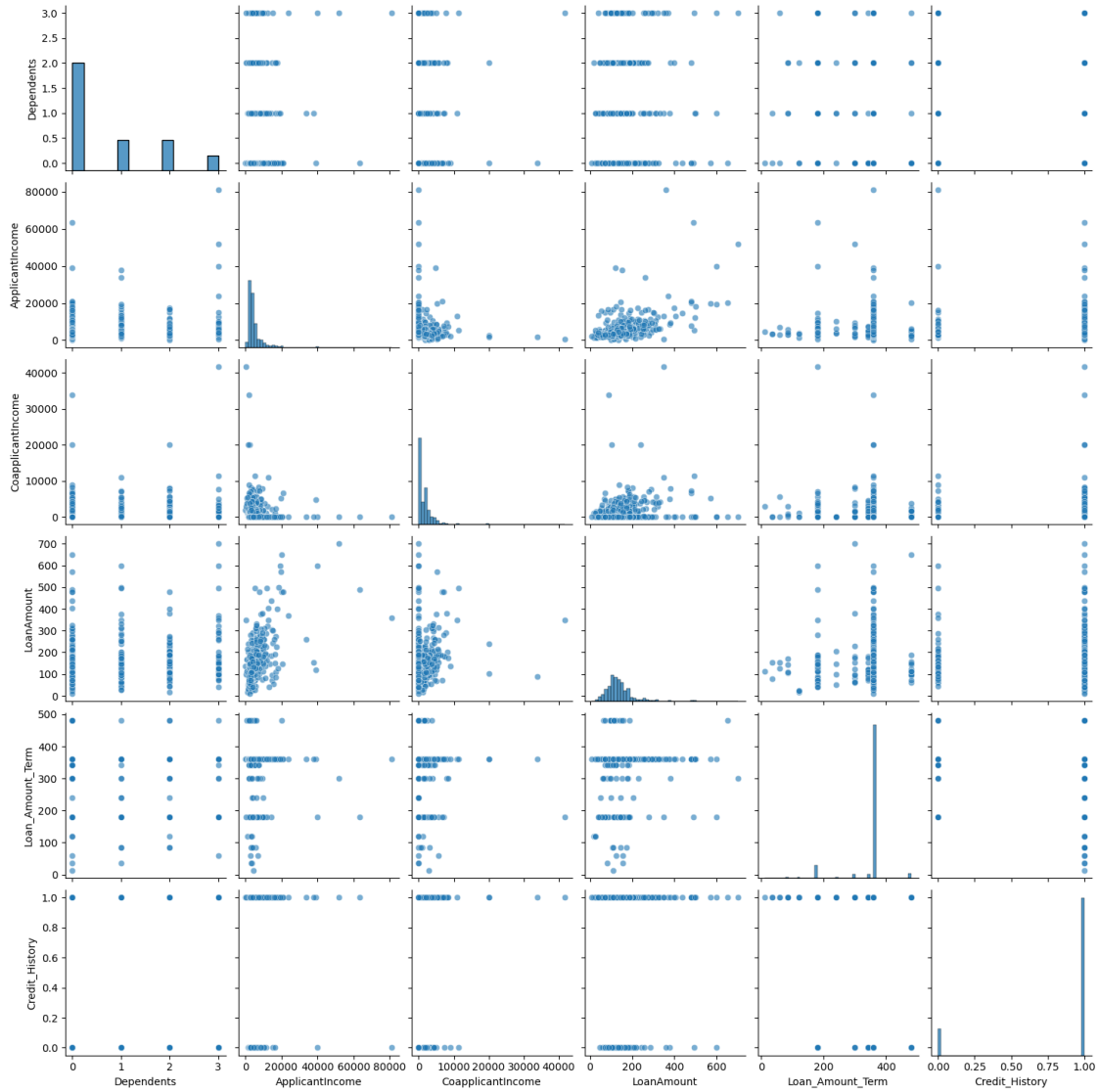
```
[22]: sns.jointplot(x = "LoanAmount", y = "ApplicantIncome", data =df, alpha = 0.5)
```

```
[22]: <seaborn.axisgrid.JointGrid at 0x2985363b620>
```



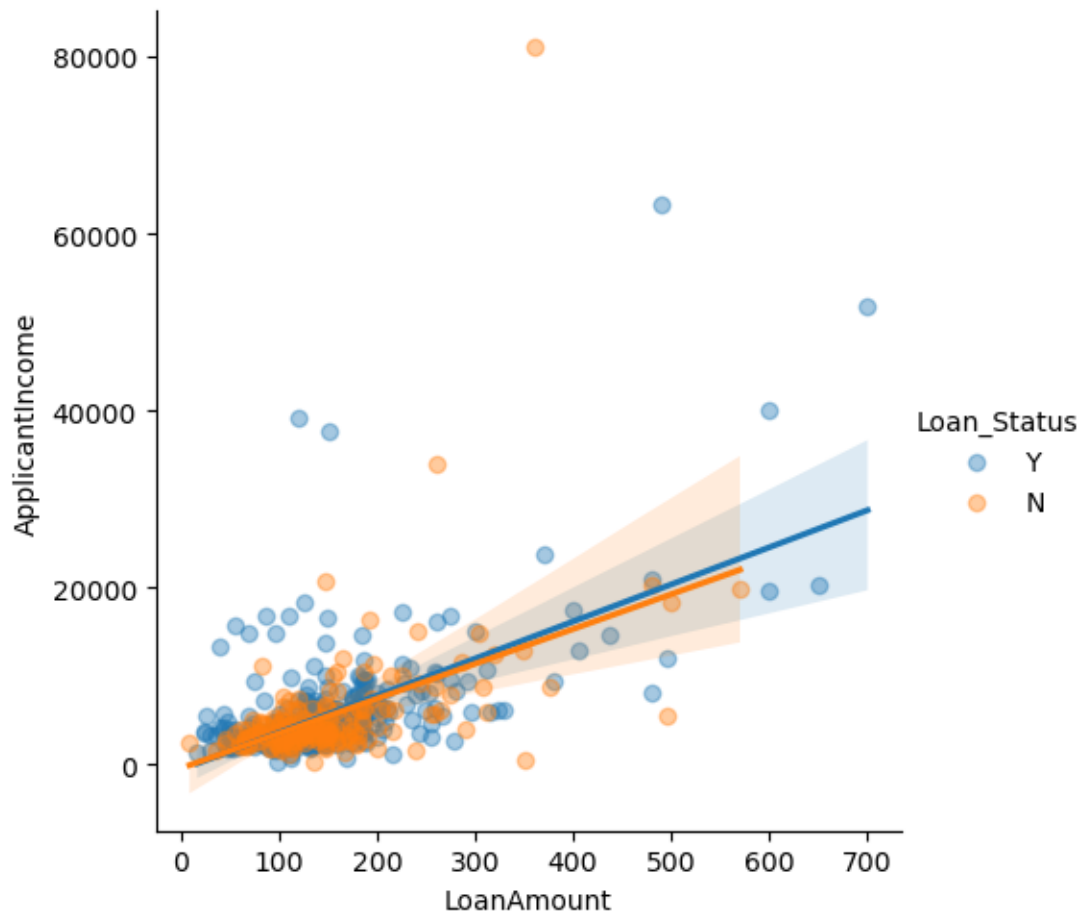
```
[23]: sns.pairplot(df, kind = "scatter", plot_kws={"alpha" : 0.6})
```

```
[23]: <seaborn.axisgrid.PairGrid at 0x2985363af90>
```



```
[24]: sns.lmplot(
    x = "LoanAmount",
    y = "ApplicantIncome",
    data = df,
    scatter_kws={"alpha" : 0.4},
    hue = "Loan_Status"
)
```

```
[24]: <seaborn.axisgrid.FacetGrid at 0x2985363b380>
```



```
[25]: df.head()
```

```
[25]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	Male	No	0.0	Graduate	No	
1	LP001003	Male	Yes	1.0	Graduate	No	
2	LP001005	Male	Yes	0.0	Graduate	Yes	
3	LP001006	Male	Yes	0.0	Not Graduate	No	
4	LP001008	Male	No	0.0	Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	146.412162	360.0	
1	4583	1508.0	128.000000	360.0	
2	3000	0.0	66.000000	360.0	
3	2583	2358.0	120.000000	360.0	
4	6000	0.0	141.000000	360.0	

	Credit_History	Property_Area	Loan_Status
0	1.0	Urban	Y

1	1.0	Rural	N
2	1.0	Urban	Y
3	1.0	Urban	Y
4	1.0	Urban	Y

```
[26]: from sklearn.cluster import KMeans
      from sklearn.preprocessing import StandardScaler
```

```
[114]: X = df[["ApplicantIncome", "LoanAmount"]]
      scaler = StandardScaler()
      X_scaled = scaler.fit_transform(X)

      model = KMeans(n_clusters = 3,
                     n_init = 10,
                     random_state = 42)
      model.fit(X_scaled)
      clusters = model.fit_predict(X_scaled)
      df["Clusters"] = clusters

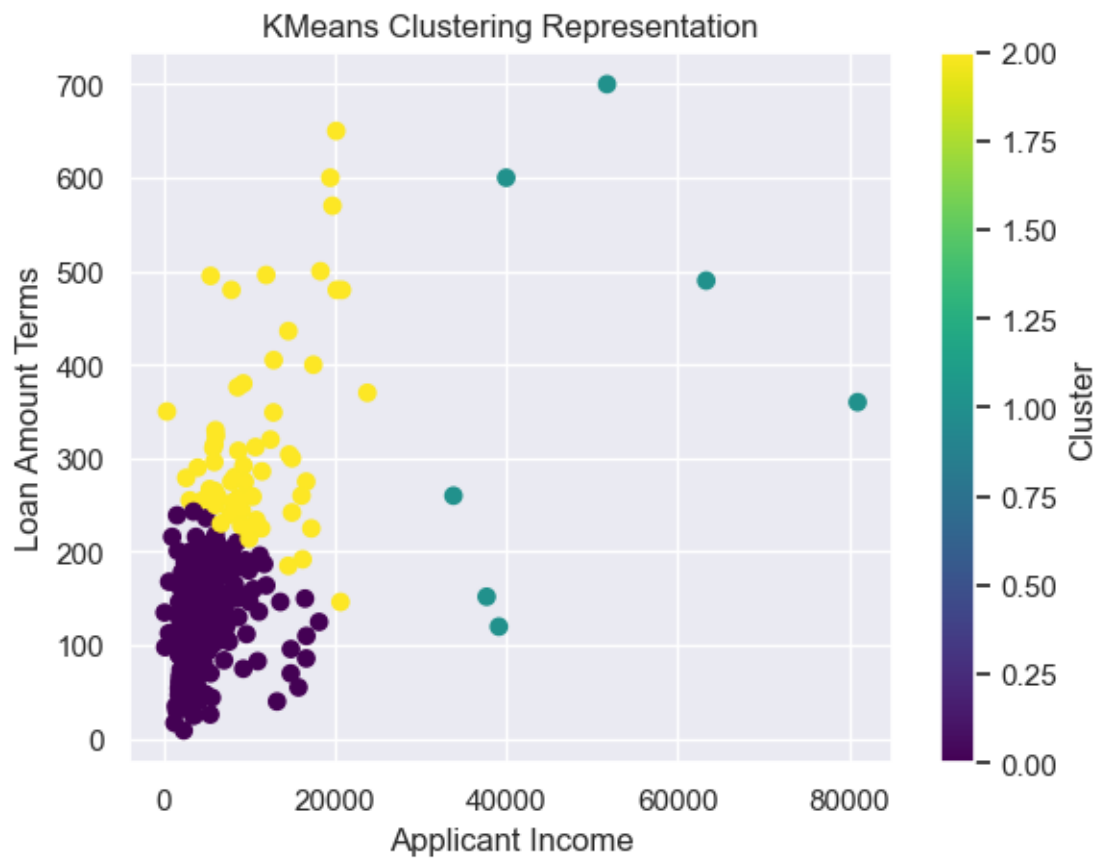
      plt.scatter(df["ApplicantIncome"],
                  df["LoanAmount"],
                  c = df["Clusters"],
                  cmap = "viridis")
      plt.title("KMeans Clustering Representation")
      plt.xlabel("Applicant Income")
      plt.ylabel("Loan Amount Terms")
      plt.colorbar(label = "Cluster")
      plt.show()
```

```
C:\Users\User\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=3.
```

```
warnings.warn(
```

```
C:\Users\User\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=3.
```

```
warnings.warn(
```



```
[28]: df["Clusters"].unique()
```

```
[28]: array([0, 2, 1], dtype=int32)
```

```
[29]: df.groupby("Clusters").agg({
    "ApplicantIncome" : ["mean", "max", "min", "std", "count", "sum"],
    "LoanAmount" : ["mean", "max", "min", "std", "count", "sum"]
})
```

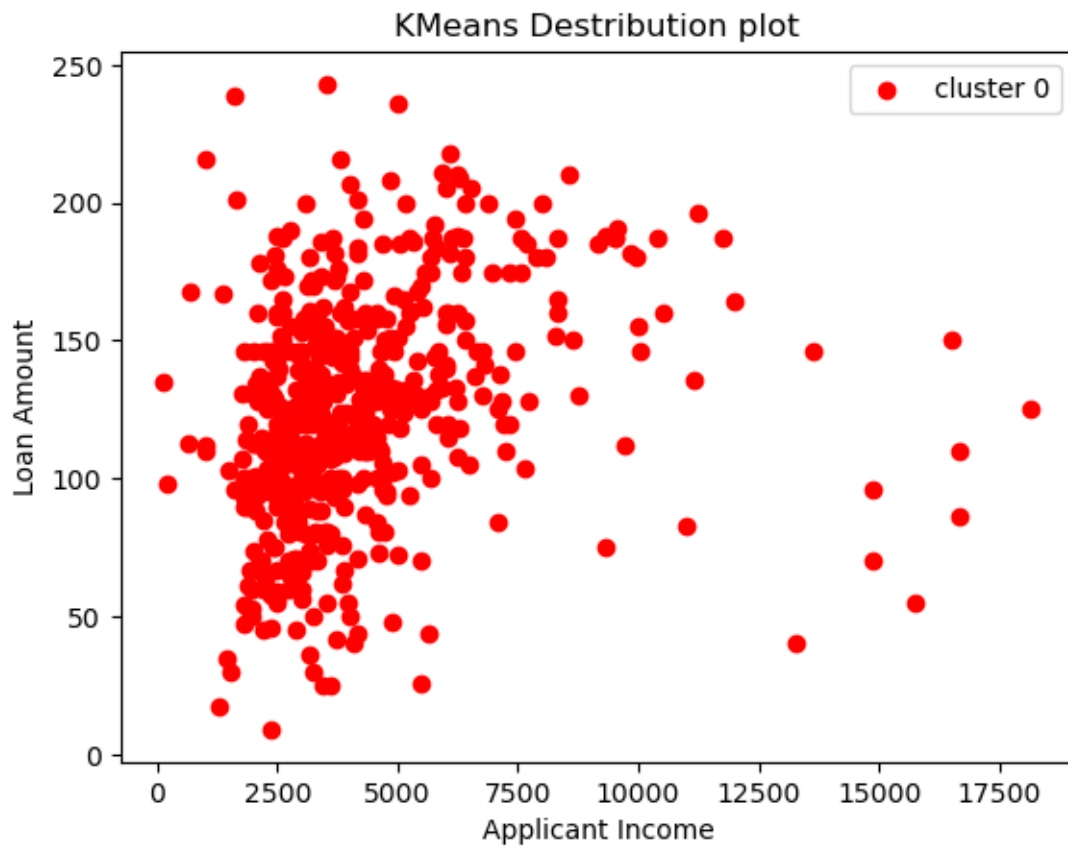
```
[29]:
```

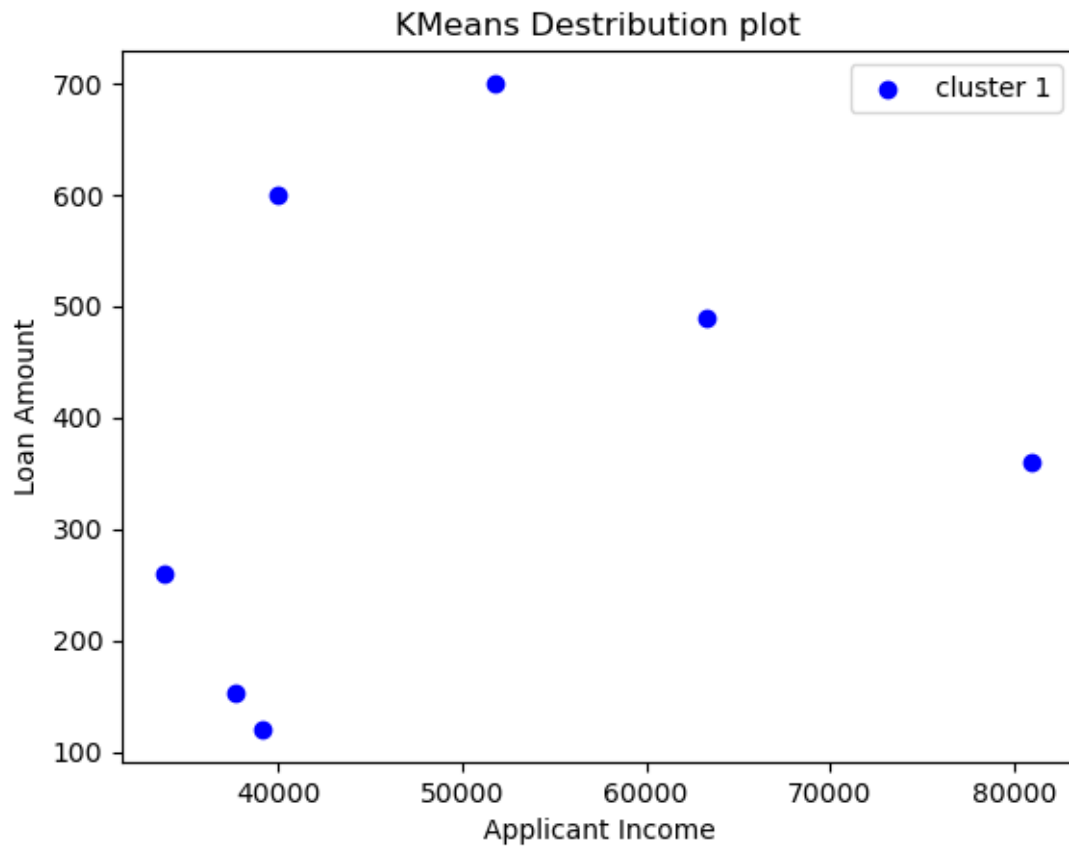
	ApplicantIncome						
	mean	max	min	std	count	sum	
Clusters							
0	4222.983486	18165	150	2418.516644	545	2301526	
1	49544.428571	81000	33846	17176.113296	7	346811	
2	10796.564516	23803	416	5315.545240	62	669387	

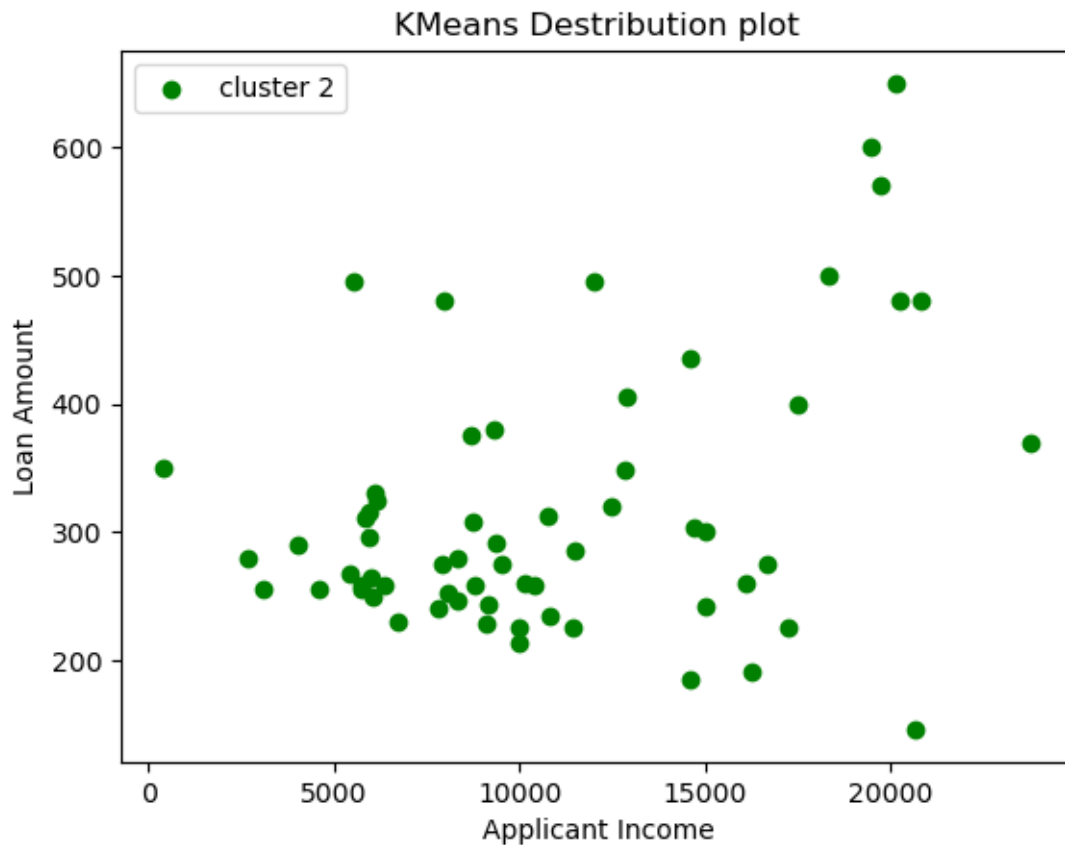
	LoanAmount						
	mean	max	min	std	count	sum	

Clusters						
0	124.028726	243.0	9.000000	40.708048	545	67595.655405
1	383.142857	700.0	120.000000	222.566416	7	2682.000000
2	316.442132	650.0	146.412162	105.014356	62	19619.412162

```
[30]: colors = ["red", "blue", "green"]
for cluster in range(3):
    data = df[df["Clusters"] == cluster]
    plt.scatter(
        data["ApplicantIncome"],
        data["LoanAmount"],
        color = colors[cluster],
        label = f"cluster {cluster}"
    )
plt.xlabel("Applicant Income")
plt.ylabel("Loan Amount")
plt.title("KMeans Distribution plot")
plt.legend()
plt.show()
```







```
[31]: from sklearn.cluster import DBSCAN
dbscan =DBSCAN(eps = 0.5,
               min_samples = 2)
dbscan.fit(X_scaled)

df["Dbscan"]= dbscan.fit_predict(X_scaled)

plt.scatter(df["ApplicantIncome"],
            df["LoanAmount"],
            c = df["Dbscan"])
plt.title("DBSACN Clustering Representation")
plt.xlabel("Applicant Income")
plt.ylabel("Loan Amount Terms")
plt.show()
```



```

0, 0, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 5, 0, 0, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 6, 0, 0, 5,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0, 0,
0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 4, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 4, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, -1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, -1, 0, 0, 0, -1, 0, 0, 0, 0, 0, 0,
0, 0])

```

```
[33]: df["Dbscan"].unique()
```

```
[33]: array([ 0, -1,  1,  2,  3,  4,  5,  6])
```

```
[34]: df.groupby("Dbscan").agg({
    "ApplicantIncome" : ["mean", "max", "min", "std", "count", "sum"],
    "LoanAmount" : ["mean", "max", "min", "std", "count", "sum", ]
})
```

```
[34]:
```

	ApplicantIncome						LoanAmount \
	mean	max	min	std	count	sum	mean
Dbscan							
-1	34383.000000	81000	416	24942.649743	10	343830	467.600000
0	4635.368866	20667	150	2953.648426	591	2739503	134.935817
1	6732.000000	7948	5516	1719.683692	2	13464	487.500000
2	38433.000000	39147	37719	1009.748484	2	76866	136.000000
3	13729.500000	14583	12876	1207.031275	2	27459	420.500000
4	19799.666667	20833	18333	1305.118130	3	59399	486.666667
5	8994.500000	9323	8666	464.569155	2	17989	378.000000
6	19607.000000	19730	19484	173.948268	2	39214	585.000000

	max	min	std	count	sum
Dbscan					
-1	700.0	260.0	144.805770	10	4676.000000
0	349.0	9.0	55.337697	591	79747.067568

1	495.0	480.0	10.606602	2	975.000000
2	152.0	120.0	22.627417	2	272.000000
3	436.0	405.0	21.920310	2	841.000000
4	500.0	480.0	11.547005	3	1460.000000
5	380.0	376.0	2.828427	2	756.000000
6	600.0	570.0	21.213203	2	1170.000000

```
[35]: from sklearn.decomposition import PCA
pca = PCA(n_components = 2)
X_pca = pca.fit_transform(X_scaled)
X_pca
```

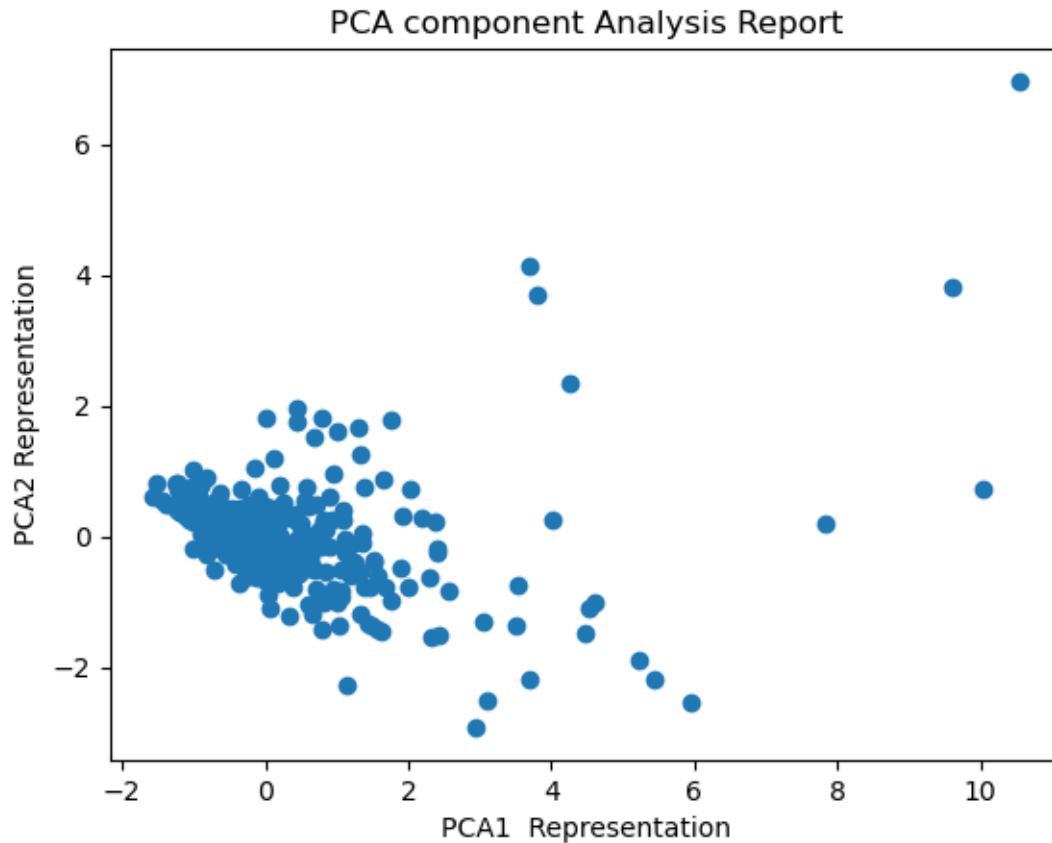
```
[35]: array([[ 0.05161231,  0.05161231],
        [-0.25009325,  0.06000604],
        [-0.95557586,  0.39873303],
        ...,
        [ 1.20670995, -0.58845203],
        [ 0.59427426, -0.08930976],
        [-0.20798803,  0.01790082]], shape=(614, 2))
```

```
[36]: df["PCA1"] = X_pca[:, 0]
df["PCA2"] = X_pca[:, 1]
```

```
[37]: plt.scatter(df["PCA1"],
                 df["PCA2"])

plt.xlabel("PCA1 Representation")
plt.ylabel("PCA2 Representation")
plt.title("PCA component Analysis Report")

plt.show()
```

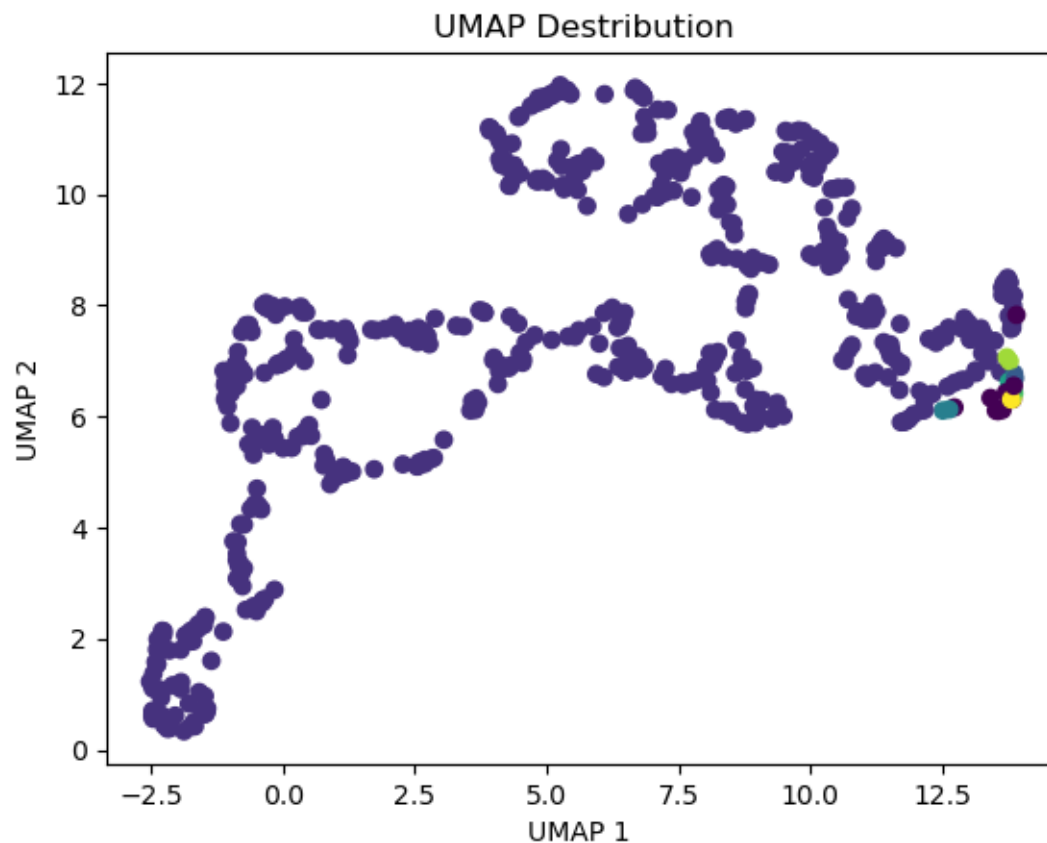


```
[38]: pca.explained_variance_ratio_.sum()
```

```
[38]: np.float64(1.0)
```

```
[39]: import umap.umap_ as umap
reducer = umap.UMAP(n_components = 2, random_state = 42)
embedding= reducer.fit_transform(X_scaled)
plt.scatter(embedding[:, 0],
            embedding[:,1],
            c = df["Dbscan"])
plt.xlabel("UMAP 1 ")
plt.ylabel("UMAP 2")
plt.title("UMAP Destribution")
plt.show()
```

C:\Users\User\anaconda3\Lib\site-packages\umap\umap_.py:1952: UserWarning:
n_jobs value 1 overridden to 1 by setting random_state. Use no seed for
parallelism.
warn(



```
[40]: df.head()
```

```
[40]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	Male	No	0.0	Graduate	No	
1	LP001003	Male	Yes	1.0	Graduate	No	
2	LP001005	Male	Yes	0.0	Graduate	Yes	
3	LP001006	Male	Yes	0.0	Not Graduate	No	
4	LP001008	Male	No	0.0	Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	146.412162	360.0	
1	4583	1508.0	128.000000	360.0	
2	3000	0.0	66.000000	360.0	
3	2583	2358.0	120.000000	360.0	
4	6000	0.0	141.000000	360.0	

	Credit_History	Property_Area	Loan_Status	Clusters	Dbscan	PCA1	\
0	1.0	Urban	Y	0	0	0.051612	
1	1.0	Rural	N	0	0	-0.250093	
2	1.0	Urban	Y	0	0	-0.955576	

3	1.0	Urban	Y	0	0	-0.549146
4	1.0	Urban	Y	0	0	0.023528

```

PCA2
0 0.051612
1 0.060006
2 0.398733
3 -0.104309
4 0.114681

```

```

[41]: from sklearn.mixture import GaussianMixture
gaussian = GaussianMixture( n_components = 3, random_state = 42)
gaussian.fit(X_scaled)
cluster = gaussian.fit_predict(X_scaled)
df["Gaussian"] = cluster
plt.scatter(df["ApplicantIncome"],
            df["LoanAmount"],
            c = df["Gaussian"] )
plt.xlabel("Applicant Income")
plt.ylabel("Loan Amount Terms")
plt.title("Gaussian Mixture Destribution Table")
plt.show()

```

```

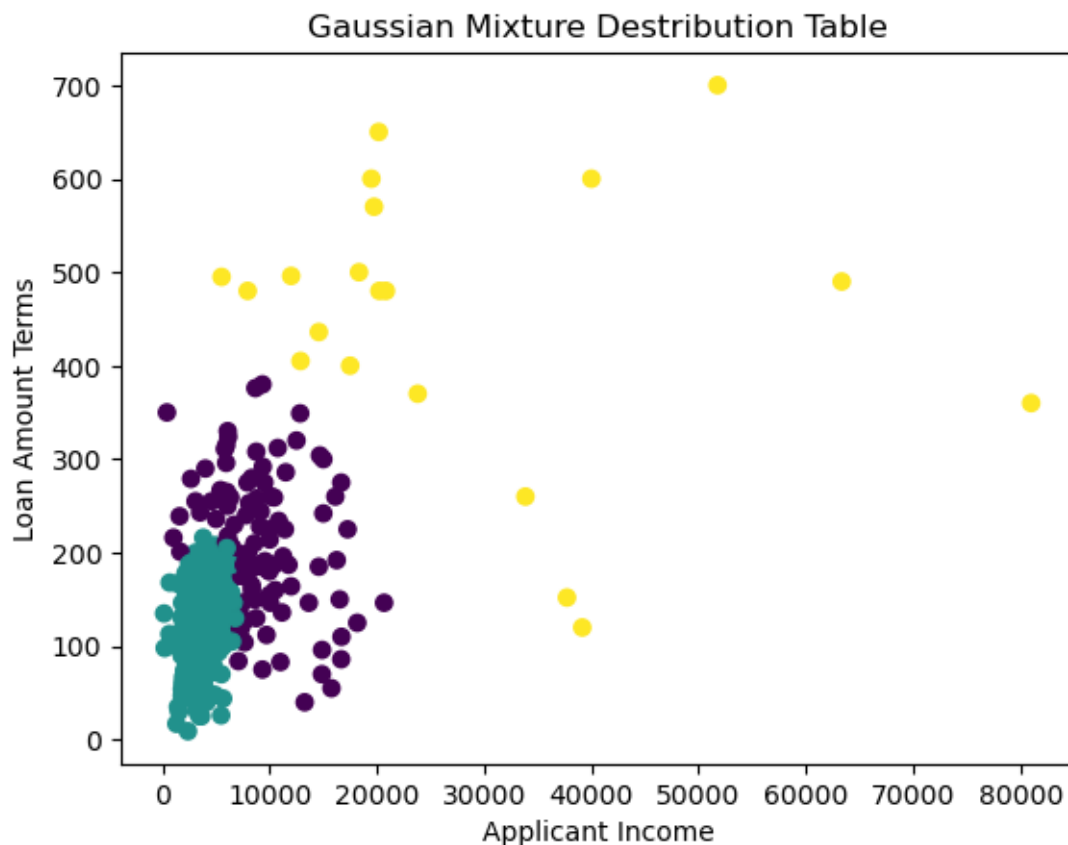
C:\Users\User\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=3.

```

```

warnings.warn(
C:\Users\User\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting the
environment variable OMP_NUM_THREADS=3.
warnings.warn(

```



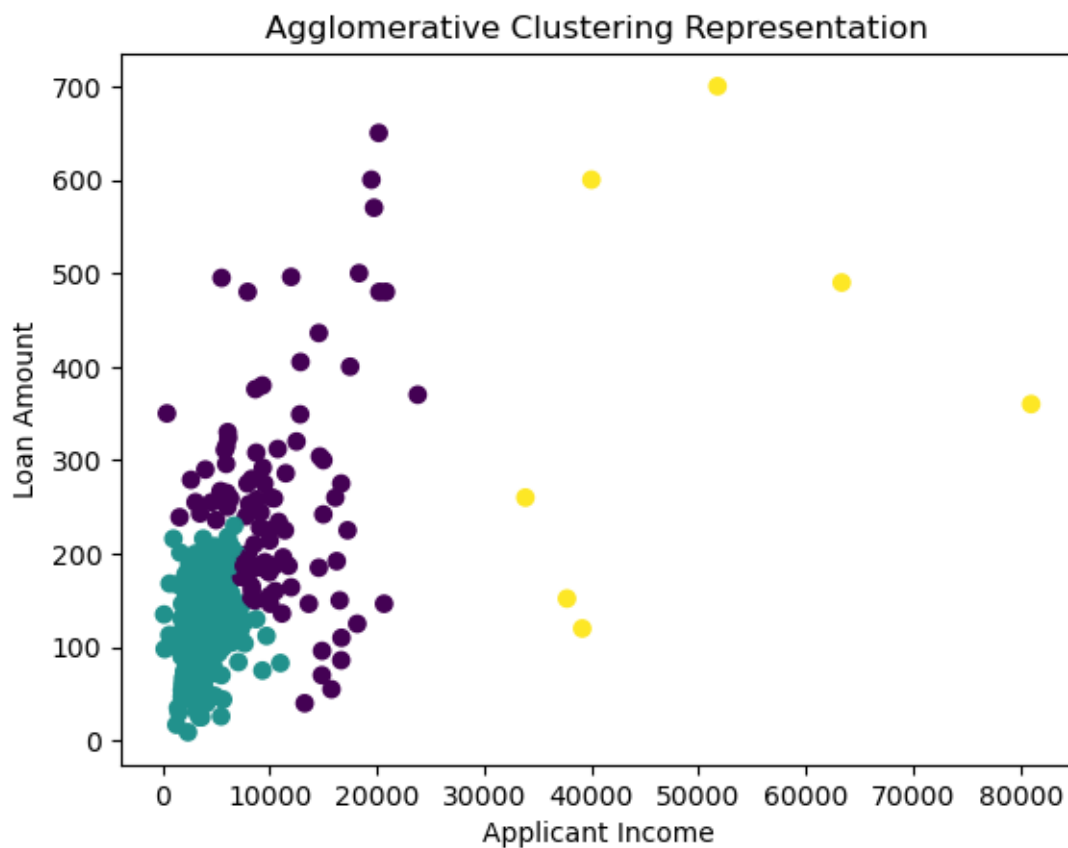
```
[42]: df.groupby("Gaussian").agg({
    "ApplicantIncome" : ["mean","max","min", "std","count", "sum"],
    "LoanAmount" : ["mean","max","min", "std","count", "sum", ]
})
```

```
[42]:
```

	ApplicantIncome					
	mean	max	min	std	count	sum
Gaussian						
0	9113.780702	20667	416	3798.140087	114	1038971
1	3581.118750	6822	150	1253.959420	480	1718937
2	27990.800000	81000	5516	19367.508256	20	559816

	LoanAmount					
	mean	max	min	std	count	sum
Gaussian						
0	207.312707	380.0	40.0	72.037257	114	23633.648649
1	119.207123	216.0	9.0	37.367137	480	57219.418919
2	452.200000	700.0	120.0	149.696043	20	9044.000000

```
[43]: from sklearn.cluster import AgglomerativeClustering
agglo = AgglomerativeClustering(n_clusters = 3 , linkage = "ward")
agglo.fit(X_scaled)
df["AgglomerativeClustering"] = agglo.fit_predict(X_scaled)
plt.scatter(df["ApplicantIncome"],
            df["LoanAmount"],
            c = df["AgglomerativeClustering"])
plt.title("Agglomerative Clustering Representation")
plt.xlabel("Applicant Income")
plt.ylabel("Loan Amount")
plt.show()
```



```
[44]: df.groupby("AgglomerativeClustering").agg({
    "ApplicantIncome" : ["mean","max","min", "std","count", "sum"],
    "LoanAmount" : ["mean", "max", "min", "std", "count", "sum", ]
})
```

```
}))
```

```
[44]:
```

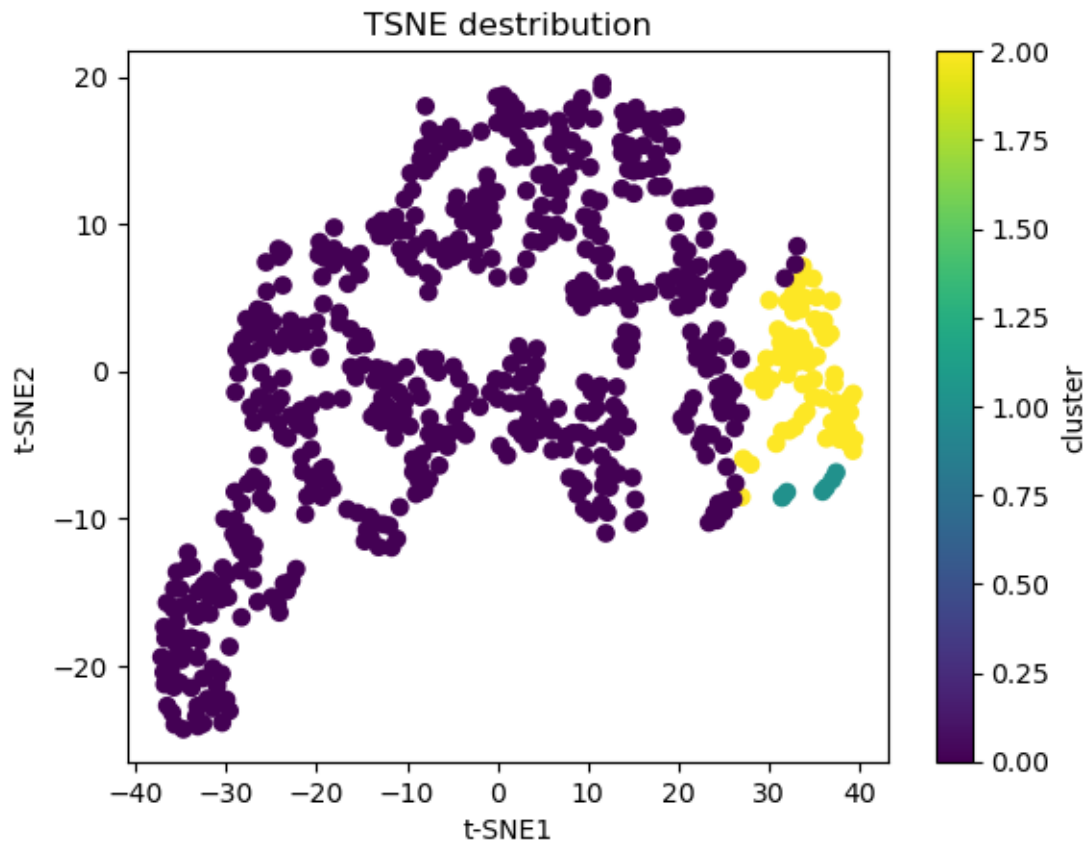
	ApplicantIncome				\	
	mean	max	min	std	count	
AgglomerativeClustering						
0	10546.718447	23803	416	4687.907720	103	
1	3739.287698	11000	150	1482.989507	504	
2	49544.428571	81000	33846	17176.113296	7	

	LoanAmount				\	
	sum	mean	max	min	std	count
AgglomerativeClustering						
0	1086312	255.264432	650.0	40.0	114.349471	103
1	1884601	120.878633	230.0	9.0	38.792263	504
2	346811	383.142857	700.0	120.0	222.566416	7

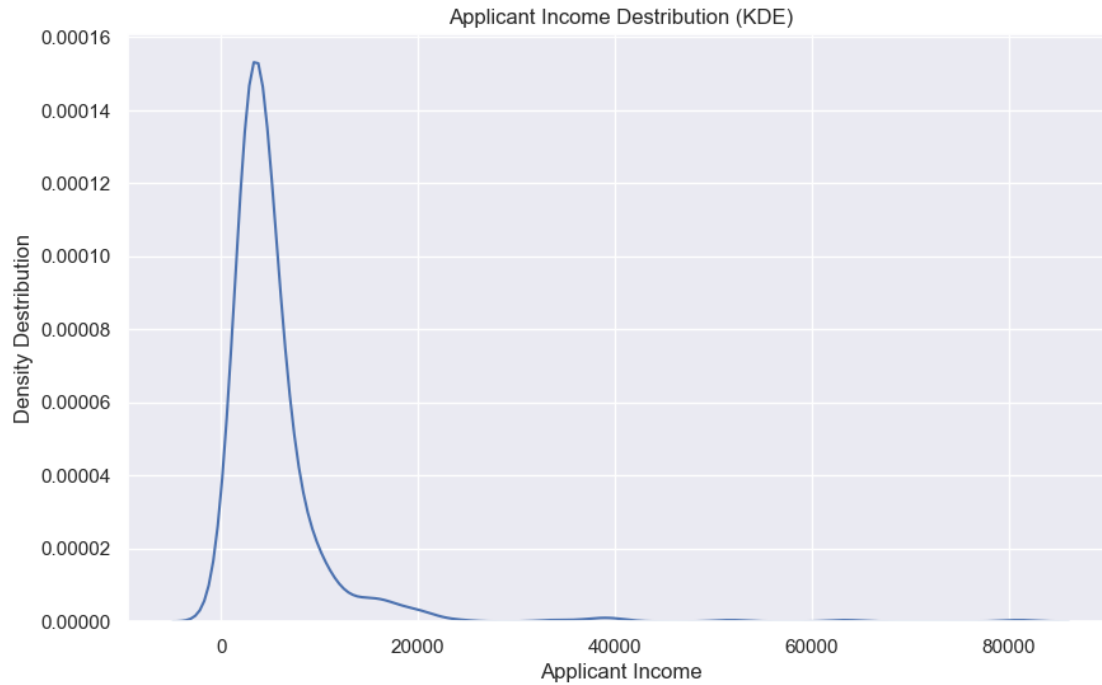
	sum
AgglomerativeClustering	
0	26292.236486
1	60922.831081
2	2682.000000

```
[45]: from sklearn.manifold import TSNE
tsne = TSNE(n_components = 2,
            random_state = 42,
            perplexity =30,
            learning_rate = "auto")

X_tsne = tsne.fit_transform(X_scaled)
plt.scatter(
    X_tsne[:, 0],
    X_tsne[:, 1],
    c = df["Clusters"],
    cmap = "viridis")
plt.xlabel("t-SNE1")
plt.ylabel("t-SNE2")
plt.title("TSNE destribution")
plt.colorbar(label =("cluster"))
plt.show()
```



```
[46]: plt.figure(figsize = (10,6))
sns.set(style = "darkgrid")
sns.kdeplot(df["ApplicantIncome"])
plt.title("Applicant Income Distribution (KDE)")
plt.xlabel("Applicant Income")
plt.ylabel("Density Distribution")
plt.show()
```



```
[47]: df.head(5)
```

```
[47]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	Male	No	0.0	Graduate	No	
1	LP001003	Male	Yes	1.0	Graduate	No	
2	LP001005	Male	Yes	0.0	Graduate	Yes	
3	LP001006	Male	Yes	0.0	Not Graduate	No	
4	LP001008	Male	No	0.0	Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	146.412162	360.0	
1	4583	1508.0	128.000000	360.0	
2	3000	0.0	66.000000	360.0	
3	2583	2358.0	120.000000	360.0	
4	6000	0.0	141.000000	360.0	

	Credit_History	Property_Area	Loan_Status	Clusters	Dbscan	PCA1	\
0	1.0	Urban	Y	0	0	0.051612	
1	1.0	Rural	N	0	0	-0.250093	
2	1.0	Urban	Y	0	0	-0.955576	
3	1.0	Urban	Y	0	0	-0.549146	
4	1.0	Urban	Y	0	0	0.023528	

	PCA2	Gaussian	AgglomerativeClustering
0			
1			
2			
3			
4			

0	0.051612	1	1
1	0.060006	1	1
2	0.398733	1	1
3	-0.104309	1	1
4	0.114681	1	1

```
[48]: df["Gender"] = df["Gender"].replace({
        "Male":1,
        "Female":0
    })
```

C:\Users\User\AppData\Local\Temp\ipykernel_15844\4026021224.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
df["Gender"] = df["Gender"].replace({
```

```
[49]: df["Self_Employed"] = df["Self_Employed"].replace({
        "Yes": 1,
        "No": 0
    })

df["Loan_Status"] = df["Loan_Status"].replace({
        "Y": 1,
        "N": 0
    })
```

C:\Users\User\AppData\Local\Temp\ipykernel_15844\80268739.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
df["Self_Employed"] = df["Self_Employed"].replace({
```

C:\Users\User\AppData\Local\Temp\ipykernel_15844\80268739.py:6: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
df["Loan_Status"] = df["Loan_Status"].replace({
```

```
[50]: df.head(20)
```

```
[50]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	1	No	0.0	Graduate	0	
1	LP001003	1	Yes	1.0	Graduate	0	
2	LP001005	1	Yes	0.0	Graduate	1	
3	LP001006	1	Yes	0.0	Not Graduate	0	

4	LP001008	1	No	0.0	Graduate	0
5	LP001011	1	Yes	2.0	Graduate	1
6	LP001013	1	Yes	0.0	Not Graduate	0
7	LP001014	1	Yes	3.0	Graduate	0
8	LP001018	1	Yes	2.0	Graduate	0
9	LP001020	1	Yes	1.0	Graduate	0
10	LP001024	1	Yes	2.0	Graduate	0
11	LP001027	1	Yes	2.0	Graduate	0
12	LP001028	1	Yes	2.0	Graduate	0
13	LP001029	1	No	0.0	Graduate	0
14	LP001030	1	Yes	2.0	Graduate	0
15	LP001032	1	No	0.0	Graduate	0
16	LP001034	1	No	1.0	Not Graduate	0
17	LP001036	0	No	0.0	Graduate	0
18	LP001038	1	Yes	0.0	Not Graduate	0
19	LP001041	1	Yes	0.0	Graduate	0

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	146.412162	360.0	
1	4583	1508.0	128.000000	360.0	
2	3000	0.0	66.000000	360.0	
3	2583	2358.0	120.000000	360.0	
4	6000	0.0	141.000000	360.0	
5	5417	4196.0	267.000000	360.0	
6	2333	1516.0	95.000000	360.0	
7	3036	2504.0	158.000000	360.0	
8	4006	1526.0	168.000000	360.0	
9	12841	10968.0	349.000000	360.0	
10	3200	700.0	70.000000	360.0	
11	2500	1840.0	109.000000	360.0	
12	3073	8106.0	200.000000	360.0	
13	1853	2840.0	114.000000	360.0	
14	1299	1086.0	17.000000	120.0	
15	4950	0.0	125.000000	360.0	
16	3596	0.0	100.000000	240.0	
17	3510	0.0	76.000000	360.0	
18	4887	0.0	133.000000	360.0	
19	2600	3500.0	115.000000	342.0	

	Credit_History	Property_Area	Loan_Status	Clusters	Dbscan	PCA1	\
0	1.0	Urban	1	0	0	0.051612	
1	1.0	Rural	0	0	0	-0.250093	
2	1.0	Urban	1	0	0	-0.955576	
3	1.0	Urban	1	0	0	-0.549146	
4	1.0	Urban	1	0	0	0.023528	
5	1.0	Urban	1	2	0	1.017044	
6	1.0	Urban	1	0	0	-0.788632	

7	0.0	Semiurban	0	0	0	-0.176669
8	1.0	Urban	1	0	0	0.019908
9	1.0	Semiurban	0	2	0	2.567581
10	1.0	Urban	1	0	0	-0.898723
11	1.0	Urban	1	0	0	-0.651392
12	1.0	Urban	1	0	0	0.181301
13	1.0	Rural	0	0	0	-0.684236
14	1.0	Urban	1	0	0	-1.565254
15	1.0	Urban	1	0	0	-0.232842
16	1.0	Urban	1	0	0	-0.600219
17	0.0	Urban	0	0	0	-0.812286
18	1.0	Rural	0	0	0	-0.172772
19	1.0	Urban	1	0	0	-0.589281

	PCA2	Gaussian	AgglomerativeClustering
0	0.051612	1	1
1	0.060006	1	1
2	0.398733	1	1
3	-0.104309	1	1
4	0.114681	1	1
5	-1.013907	0	0
6	0.077256	1	1
7	-0.371833	1	1
8	-0.343677	1	1
9	-0.844422	0	0
10	0.388217	1	1
11	-0.021293	1	1
12	-0.721231	1	1
13	-0.138348	1	1
14	0.614317	1	1
15	0.127783	1	1
16	0.181459	1	1
17	0.373602	1	1
18	0.053117	1	1
19	-0.060235	1	1

```
[51]: df["LoanAmount"].unique().sum()
```

```
[51]: np.float64(36197.41216216216)
```

```
[52]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               614 non-null    object
```

```

1  Gender                614 non-null    int64
2  Married               614 non-null    object
3  Dependents            614 non-null    float64
4  Education             614 non-null    object
5  Self_Employed         614 non-null    int64
6  ApplicantIncome       614 non-null    int64
7  CoapplicantIncome     614 non-null    float64
8  LoanAmount            614 non-null    float64
9  Loan_Amount_Term      614 non-null    float64
10 Credit_History        614 non-null    float64
11 Property_Area         614 non-null    object
12 Loan_Status           614 non-null    int64
13 Clusters              614 non-null    int32
14 Dbscan                614 non-null    int64
15 PCA1                  614 non-null    float64
16 PCA2                  614 non-null    float64
17 Gaussian              614 non-null    int64
18 AgglomerativeClustering 614 non-null    int64
dtypes: float64(7), int32(1), int64(7), object(4)
memory usage: 88.9+ KB

```

```
[53]: df = df.dropna(subset = "LoanAmount")
```

```
[54]: from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
[55]: df.head()
```

```
[55]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	1	No	0.0	Graduate	0	
1	LP001003	1	Yes	1.0	Graduate	0	
2	LP001005	1	Yes	0.0	Graduate	1	
3	LP001006	1	Yes	0.0	Not Graduate	0	
4	LP001008	1	No	0.0	Graduate	0	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	146.412162	360.0	
1	4583	1508.0	128.000000	360.0	
2	3000	0.0	66.000000	360.0	
3	2583	2358.0	120.000000	360.0	
4	6000	0.0	141.000000	360.0	

	Credit_History	Property_Area	Loan_Status	Clusters	Dbscan	PCA1	\
0	1.0	Urban	1	0	0	0.051612	
1	1.0	Rural	0	0	0	-0.250093	
2	1.0	Urban	1	0	0	-0.955576	
3	1.0	Urban	1	0	0	-0.549146	

4	1.0	Urban	1	0	0	0.023528
---	-----	-------	---	---	---	----------

	PCA2	Gaussian	AgglomerativeClustering
0	0.051612	1	1
1	0.060006	1	1
2	0.398733	1	1
3	-0.104309	1	1
4	0.114681	1	1

```
[56]: from sklearn.metrics import mean_squared_error, mean_absolute_error
```

```
[ ]:
```

```
[57]: type(X)
```

```
[57]: pandas.core.frame.DataFrame
```

```
[58]: df["Loan_Status"].unique()
```

```
[58]: array([1, 0])
```

```
[59]: type(X)
```

```
[59]: pandas.core.frame.DataFrame
```

```
[60]: df.columns
```

```
[60]: Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
        'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
        'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status',
        'Clusters', 'Dbscan', 'PCA1', 'PCA2', 'Gaussian',
        'AgglomerativeClustering'],
        dtype='object')
```

```
[61]: df.head(5)
```

```
[61]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	1	No	0.0	Graduate	0	
1	LP001003	1	Yes	1.0	Graduate	0	
2	LP001005	1	Yes	0.0	Graduate	1	
3	LP001006	1	Yes	0.0	Not Graduate	0	
4	LP001008	1	No	0.0	Graduate	0	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	146.412162	360.0	
1	4583	1508.0	128.000000	360.0	
2	3000	0.0	66.000000	360.0	
3	2583	2358.0	120.000000	360.0	

4	6000	0.0	141.000000	360.0
---	------	-----	------------	-------

	Credit_History	Property_Area	Loan_Status	Clusters	Dbscan	PCA1 \
0	1.0	Urban	1	0	0	0.051612
1	1.0	Rural	0	0	0	-0.250093
2	1.0	Urban	1	0	0	-0.955576
3	1.0	Urban	1	0	0	-0.549146
4	1.0	Urban	1	0	0	0.023528

	PCA2	Gaussian	AgglomerativeClustering
0	0.051612	1	1
1	0.060006	1	1
2	0.398733	1	1
3	-0.104309	1	1
4	0.114681	1	1

```
[62]: print(df.dtypes)
```

```
Loan_ID          object
Gender           int64
Married          object
Dependents       float64
Education        object
Self_Employed    int64
ApplicantIncome  int64
CoapplicantIncome float64
LoanAmount       float64
Loan_Amount_Term float64
Credit_History  float64
Property_Area    object
Loan_Status      int64
Clusters         int32
Dbscan           int64
PCA1             float64
PCA2             float64
Gaussian         int64
AgglomerativeClustering int64
dtype: object
```

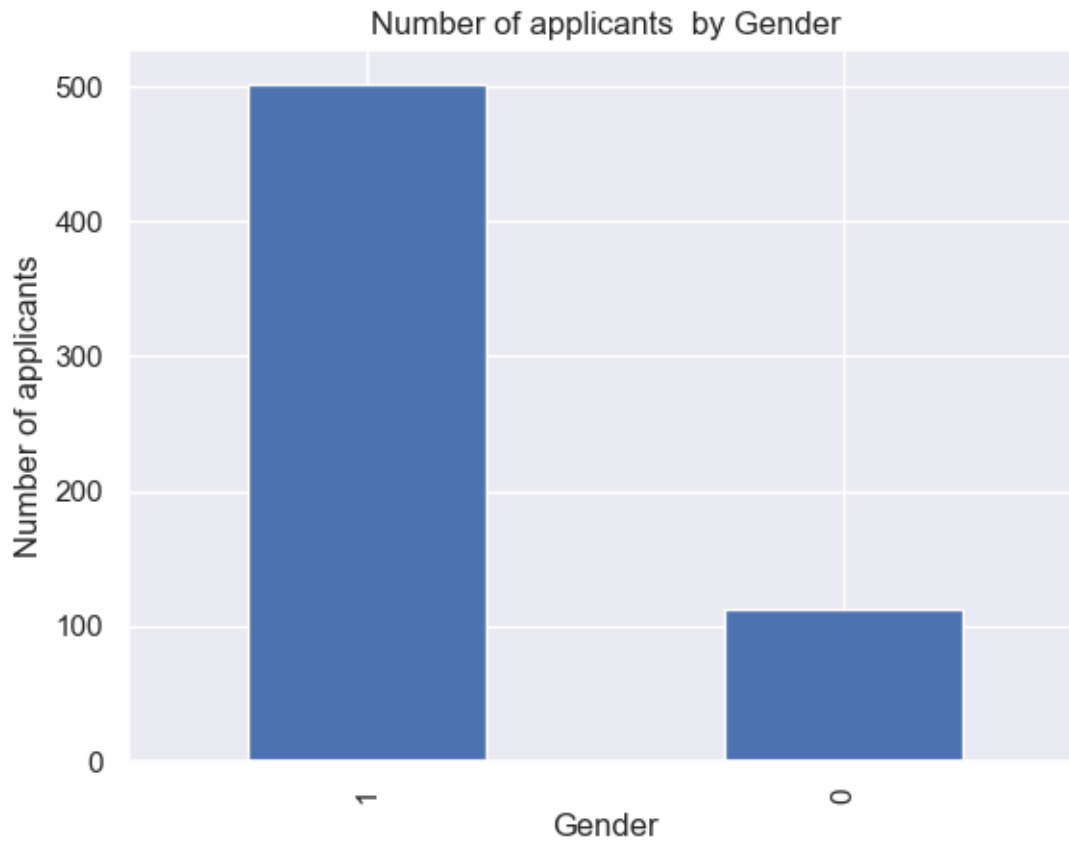
```
[63]: df["Credit_History"].unique()
```

```
[63]: array([1., 0.])
```

```
[64]: print(df["Gender"].unique())
```

```
[1 0]
```

```
[65]: df["Gender"].value_counts().plot(kind="bar")
plt.title("Number of applicants by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of applicants")
plt.show()
```



```
[66]: df.head(100)
```

```
[66]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001002	1	No	0.0	Graduate	0	
1	LP001003	1	Yes	1.0	Graduate	0	
2	LP001005	1	Yes	0.0	Graduate	1	
3	LP001006	1	Yes	0.0	Not Graduate	0	
4	LP001008	1	No	0.0	Graduate	0	
..	...	...	...	...	...	...	
95	LP001326	1	No	0.0	Graduate	0	
96	LP001327	0	Yes	0.0	Graduate	0	
97	LP001333	1	Yes	0.0	Graduate	0	
98	LP001334	1	Yes	0.0	Not Graduate	0	
99	LP001343	1	Yes	0.0	Graduate	0	

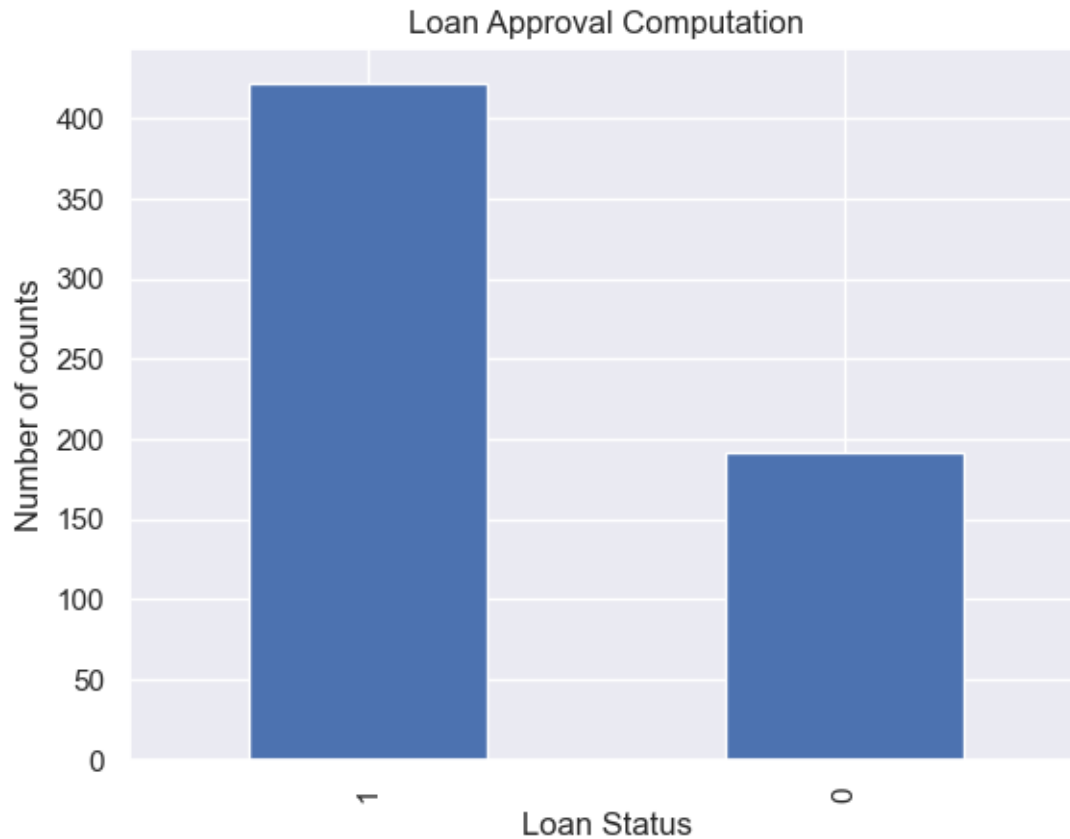
	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5849	0.0	146.412162	360.0	
1	4583	1508.0	128.000000	360.0	
2	3000	0.0	66.000000	360.0	
3	2583	2358.0	120.000000	360.0	
4	6000	0.0	141.000000	360.0	
..	...	...	...	...	
95	6782	0.0	146.412162	360.0	
96	2484	2302.0	137.000000	360.0	
97	1977	997.0	50.000000	360.0	
98	4188	0.0	115.000000	180.0	
99	1759	3541.0	131.000000	360.0	

	Credit_History	Property_Area	Loan_Status	Clusters	Dbscan	PCA1	\
0	1.0	Urban	1	0	0	0.051612	
1	1.0	Rural	0	0	0	-0.250093	
2	1.0	Urban	1	0	0	-0.955576	
3	1.0	Urban	1	0	0	-0.549146	
4	1.0	Urban	1	0	0	0.023528	
..	...	...	...	...	...	...	
95	1.0	Urban	0	0	0	0.159693	
96	1.0	Semiurban	1	0	0	-0.417456	
97	1.0	Semiurban	1	0	0	-1.208819	
98	1.0	Semiurban	1	0	0	-0.405324	
99	1.0	Semiurban	1	0	0	-0.551968	

	PCA2	Gaussian	AgglomerativeClustering
0	0.051612	1	1
1	0.060006	1	1
2	0.398733	1	1
3	-0.104309	1	1
4	0.114681	1	1
..	...	...	...
95	0.159693	1	1
96	-0.258936	1	1
97	0.414963	1	1
98	0.123722	1	1
99	-0.292395	1	1

[100 rows x 19 columns]

```
[67]: df["Loan_Status"].value_counts().plot(kind="bar")
plt.title("Loan Approval Computation")
plt.xlabel("Loan Status")
plt.ylabel("Number of counts")
plt.show()
```



3.1 Interpretation

The bar chart shows that loan approvals (Y) are higher than loan rejections (N). This indicates that the bank approved most of the loan applications in the dataset.

3.2 Business Insight

The high approval rate suggests that a large proportion of applicants met the bank's lending requirements.

3.3 Recommendation

Further analysis should be conducted to identify the factors contributing to loan approvals, such as credit history, income, and loan amount.

```
[68]: df.columns
```

```
[68]: Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
        'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
        'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status',
        'Clusters', 'Dbscan', 'PCA1', 'PCA2', 'Gaussian',
```

```
'AgglomerativeClustering'],  
dtype='object')
```

```
[69]: pd.crosstab(df["Gender"],df["Loan_Status"])
```

```
[69]: Loan_Status    0    1  
Gender  
0           37   75  
1          155  347
```

```
[70]: pd.crosstab(df["Gender"],df["Loan_Status"]).plot(kind = "bar")  
plt.title("Loan Status by Gender")  
plt.xlabel("Gender")  
plt.ylabel("Loan Status Approval")  
plt.show()
```



```
[71]: pd.crosstab(df["Married"],df["Loan_Status"]).plot(kind = "bar")  
plt.title("Loan Status by Gender")  
plt.xlabel("Married")  
plt.ylabel("Loan Status Approval")
```

```
plt.show()
```

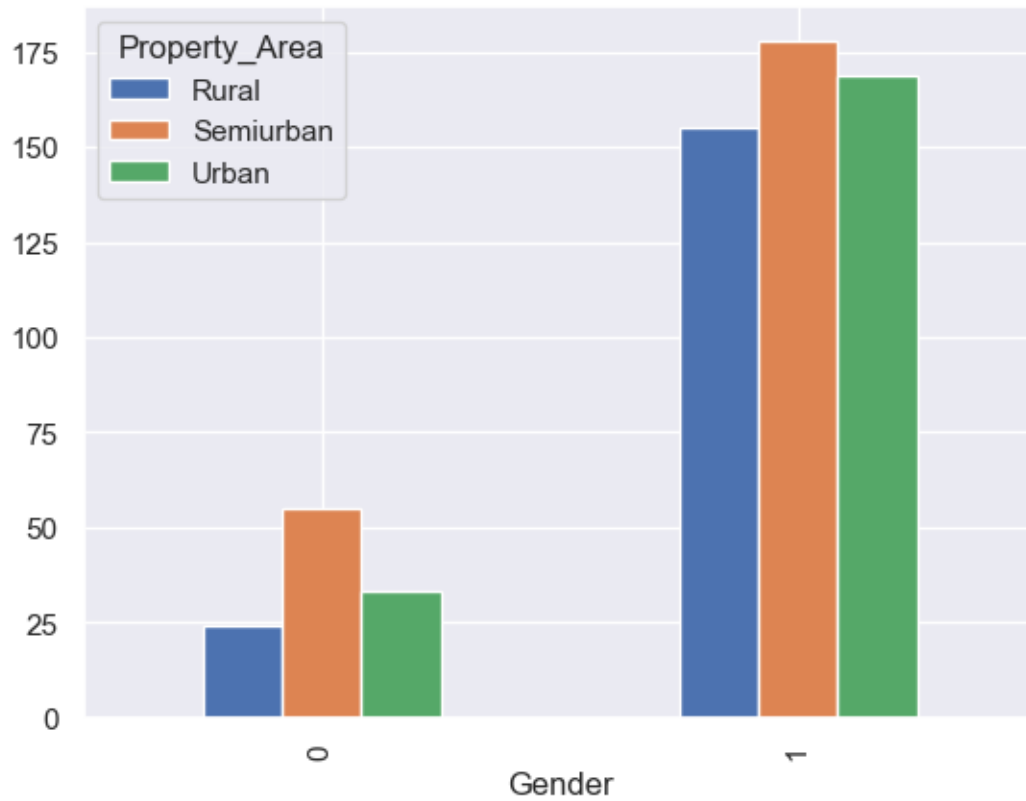


```
[72]: pd.crosstab(df["Gender"],df["Property_Area"])
```

```
[72]: Property_Area  Rural  Semiurban  Urban
Gender
0           24      55      33
1          155     178     169
```

```
[73]: data=pd.crosstab(df["Gender"],df["Property_Area"]).plot(kind="bar")
print(data)
```

Axes(0.125,0.11;0.775x0.77)



```
[74]: pd.set_option("display.max_rows",614)
```

```
[75]: plt.figure(figsize=(8,5))

plt.scatter(df["ApplicantIncome"], df["LoanAmount"])

plt.title("Applicant Income vs Loan Amount")
plt.xlabel("Applicant Income")
plt.ylabel("Loan Amount")

plt.show()
```



```
[76]: corr=df.corr(numeric_only =True)
      corr
```

```
[76]:
```

	Gender	Dependents	Self_Employed	ApplicantIncome	\
Gender	1.000000	0.172914	-0.000525	0.058809	
Dependents	0.172914	1.000000	0.056798	0.118202	
Self_Employed	-0.000525	0.056798	1.000000	0.127180	
ApplicantIncome	0.058809	0.118202	0.127180	1.000000	
CoapplicantIncome	0.082912	0.030430	-0.016100	-0.116605	
LoanAmount	0.107930	0.163106	0.115260	0.565620	
Loan_Amount_Term	-0.073567	-0.101054	-0.033943	-0.045242	
Credit_History	0.009170	-0.040160	-0.001550	-0.018615	
Loan_Status	0.017987	0.010118	-0.003700	-0.004710	
Clusters	0.027018	0.099550	0.090582	0.428998	
Dbscan	0.018694	-0.038460	0.100806	0.126270	
PCA1	0.094228	0.158973	0.137008	0.884766	
PCA2	-0.052701	-0.048176	0.012788	0.466036	
Gaussian	-0.010956	-0.022078	-0.145861	0.017392	
AgglomerativeClustering	-0.005483	-0.043292	-0.172597	-0.149760	

	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
Gender	0.082912	0.107930	-0.073567	
Dependents	0.030430	0.163106	-0.101054	

Self_Employed	-0.016100	0.115260	-0.033943
ApplicantIncome	-0.116605	0.565620	-0.045242
CoapplicantIncome	1.000000	0.187828	-0.059675
LoanAmount	0.187828	1.000000	0.038801
Loan_Amount_Term	-0.059675	0.038801	1.000000
Credit_History	0.011134	-0.001431	0.000432
Loan_Status	-0.059187	-0.036416	-0.020974
Clusters	0.120739	0.725340	0.035727
Dbscan	0.048556	0.359574	0.036206
PCA1	0.040250	0.884766	-0.003640
PCA2	-0.326619	-0.466036	-0.090168
Gaussian	-0.042112	-0.036339	0.028915
AgglomerativeClustering	-0.091989	-0.471165	-0.032832

	Credit_History	Loan_Status	Clusters	Dbscan	\
Gender	0.009170	0.017987	0.027018	0.018694	
Dependents	-0.040160	0.010118	0.099550	-0.038460	
Self_Employed	-0.001550	-0.003700	0.090582	0.100806	
ApplicantIncome	-0.018615	-0.004710	0.428998	0.126270	
CoapplicantIncome	0.011134	-0.059187	0.120739	0.048556	
LoanAmount	-0.001431	-0.036416	0.725340	0.359574	
Loan_Amount_Term	0.000432	-0.020974	0.035727	0.036206	
Credit_History	1.000000	0.540556	0.037951	0.009733	
Loan_Status	0.540556	1.000000	-0.052202	-0.034840	
Clusters	0.037951	-0.052202	1.000000	0.302108	
Dbscan	0.009733	-0.034840	0.302108	1.000000	
PCA1	-0.011328	-0.023241	0.652341	0.274561	
PCA2	-0.018437	0.034017	-0.317939	-0.250307	
Gaussian	-0.038000	0.004822	-0.272731	0.149823	
AgglomerativeClustering	-0.081336	0.008757	-0.643596	-0.231475	

	PCA1	PCA2	Gaussian	\
Gender	9.422767e-02	-5.270068e-02	-0.010956	
Dependents	1.589728e-01	-4.817618e-02	-0.022078	
Self_Employed	1.370078e-01	1.278812e-02	-0.145861	
ApplicantIncome	8.847656e-01	4.660362e-01	0.017392	
CoapplicantIncome	4.025010e-02	-3.266194e-01	-0.042112	
LoanAmount	8.847656e-01	-4.660362e-01	-0.036339	
Loan_Amount_Term	-3.639578e-03	-9.016820e-02	0.028915	
Credit_History	-1.132841e-02	-1.843719e-02	-0.038000	
Loan_Status	-2.324092e-02	3.401719e-02	0.004822	
Clusters	6.523414e-01	-3.179394e-01	-0.272731	
Dbscan	2.745608e-01	-2.503073e-01	0.149823	
PCA1	1.000000e+00	4.044019e-17	-0.010707	
PCA2	4.044019e-17	1.000000e+00	0.057647	
Gaussian	-1.070707e-02	5.764690e-02	1.000000	
AgglomerativeClustering	-3.508978e-01	3.448286e-01	0.650175	

	AgglomerativeClustering
Gender	-0.005483
Dependents	-0.043292
Self_Employed	-0.172597
ApplicantIncome	-0.149760
CoapplicantIncome	-0.091989
LoanAmount	-0.471165
Loan_Amount_Term	-0.032832
Credit_History	-0.081336
Loan_Status	0.008757
Clusters	-0.643596
Dbscan	-0.231475
PCA1	-0.350898
PCA2	0.344829
Gaussian	0.650175
AgglomerativeClustering	1.000000

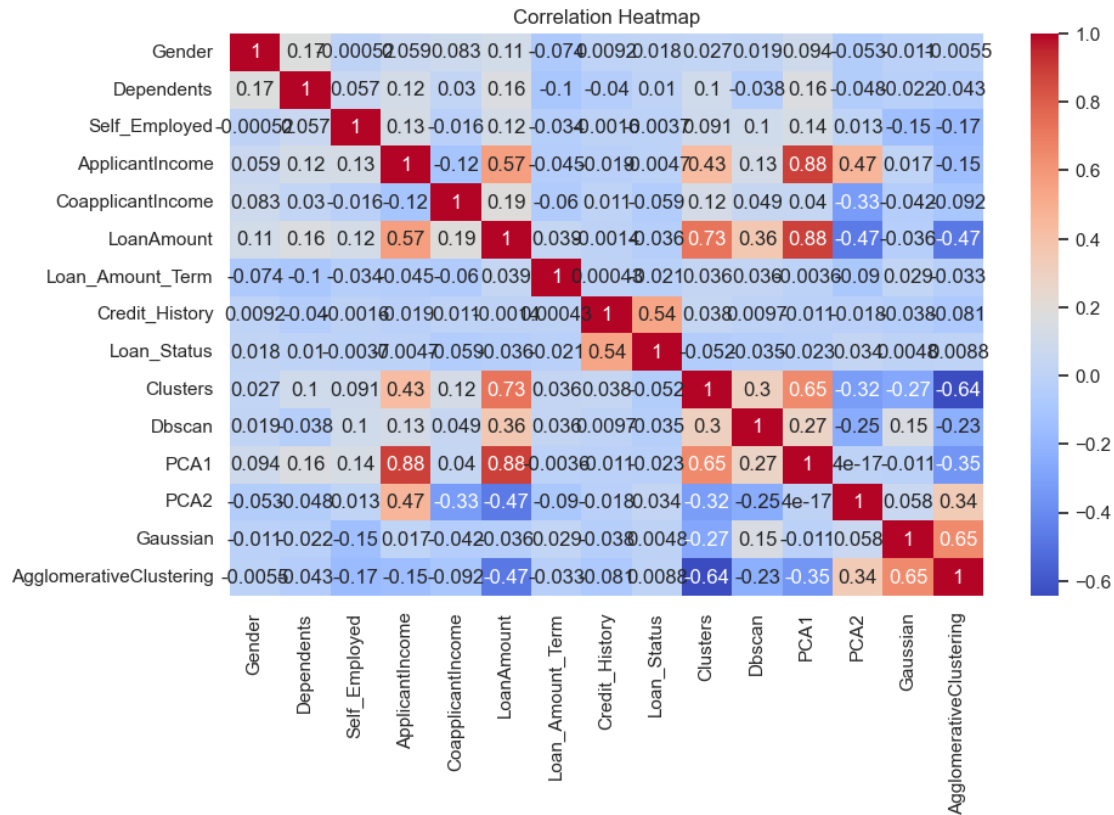
```
[77]: plt.figure(figsize=(10,6))

corr = df.corr(numeric_only=True)

sns.heatmap(corr,
            annot=True,
            cmap="coolwarm")

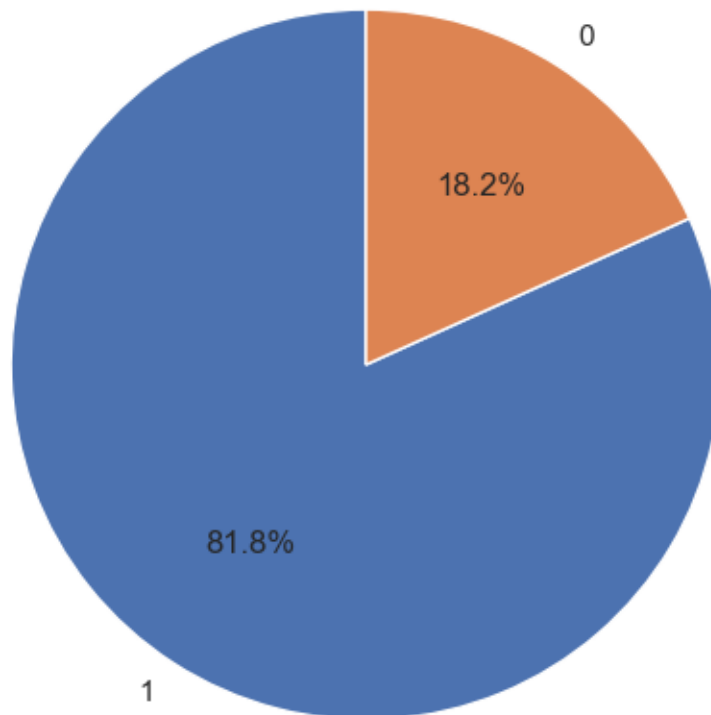
plt.title("Correlation Heatmap")

plt.show()
```



```
[78]: plt.figure(figsize=(6,6))
df["Gender"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Gender computation")
plt.ylabel("")
plt.show()
```

Gender computation



```
[79]: df.isnull().sum()
```

```
[79]: Loan_ID          0
      Gender         0
      Married       0
      Dependents    0
      Education     0
      Self_Employed 0
      ApplicantIncome 0
      CoapplicantIncome 0
      LoanAmount    0
      Loan_Amount_Term 0
      Credit_History 0
      Property_Area  0
      Loan_Status    0
      Clusters       0
      Dbscan         0
```

```

PCA1                0
PCA2                0
Gaussian            0
AgglomerativeClustering 0
dtype: int64

```

```
[80]: df.head()
```

```

[80]:   Loan_ID  Gender  Married  Dependents  Education  Self_Employed  \
0  LP001002      1      No        0.0    Graduate           0
1  LP001003      1     Yes        1.0    Graduate           0
2  LP001005      1     Yes        0.0    Graduate           1
3  LP001006      1     Yes        0.0  Not Graduate           0
4  LP001008      1      No        0.0    Graduate           0

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0           5849           0.0  146.412162           360.0
1           4583          1508.0  128.000000           360.0
2           3000           0.0   66.000000           360.0
3           2583          2358.0  120.000000           360.0
4           6000           0.0  141.000000           360.0

   Credit_History  Property_Area  Loan_Status  Clusters  Dbscan  PCA1  \
0             1.0          Urban            1         0        0  0.051612
1             1.0          Rural            0         0        0 -0.250093
2             1.0          Urban            1         0        0 -0.955576
3             1.0          Urban            1         0        0 -0.549146
4             1.0          Urban            1         0        0  0.023528

   PCA2  Gaussian  AgglomerativeClustering
0  0.051612      1                      1
1  0.060006      1                      1
2  0.398733      1                      1
3 -0.104309      1                      1
4  0.114681      1                      1

```

```
[81]: df.shape
```

```
[81]: (614, 19)
```

```
[82]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               614 non-null   object

```

```

1  Gender                614 non-null    int64
2  Married                614 non-null    object
3  Dependents            614 non-null    float64
4  Education             614 non-null    object
5  Self_Employed         614 non-null    int64
6  ApplicantIncome       614 non-null    int64
7  CoapplicantIncome     614 non-null    float64
8  LoanAmount            614 non-null    float64
9  Loan_Amount_Term      614 non-null    float64
10 Credit_History        614 non-null    float64
11 Property_Area         614 non-null    object
12 Loan_Status           614 non-null    int64
13 Clusters              614 non-null    int32
14 Dbscan                614 non-null    int64
15 PCA1                  614 non-null    float64
16 PCA2                  614 non-null    float64
17 Gaussian              614 non-null    int64
18 AgglomerativeClustering 614 non-null    int64
dtypes: float64(7), int32(1), int64(7), object(4)
memory usage: 88.9+ KB

```

```
[83]: pd.set_option("display.max_columns", 614)
```

4 SUPERVISED MACHINE LEARNING MODEL

- 1.Logistics Regression
- 2.Linear Regression
- 3.Naive Bayes (GaussianNB) destribution
- 4.Random Forest Classifiers
- 5.SVC (Surpot Vector Machine)
6. Xgboost Classifiers
- 7.Decission Tree Classifiers

4.0.1 NB

-We will take the most performed machine learning model with the highest accuracy score to make the final predictions

```
[84]: X_
      ↪=df[["ApplicantIncome","CoapplicantIncome","LoanAmount","Loan_Amount_Term","Credit_History"]
      ↪# feature variables
y =df["Loan_Status"] # target Variabe(either approved or rejected)
```

```
[85]: X.shape
```

```
[85]: (614, 5)
```

```
[86]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test =train_test_split(X,
                                                y,
                                                test_size = 0.2,
                                                random_state = 42)
```

```
[87]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_trained_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
[88]: X_test_scaled
```

```
[88]: array([[ 6.04084831e-01, -5.28127061e-01,  9.20698112e-01,
               2.94492056e-01,  4.07763147e-01],
              [-2.01225648e-01, -5.28127061e-01, -2.16350571e-01,
               2.94492056e-01,  4.07763147e-01],
              [-2.25352905e-01, -7.22934567e-02,  1.08520481e-01,
               2.94492056e-01,  4.07763147e-01],
              [-4.39123777e-01, -5.28127061e-01, -9.00900289e-01,
               2.94492056e-01,  4.07763147e-01],
              [-4.69831196e-01, -5.28127061e-01, -9.12502826e-01,
               2.94492056e-01,  4.07763147e-01],
              [-6.27755061e-01,  1.74997061e-01, -4.83208936e-01,
               2.94492056e-01,  4.07763147e-01],
              [-3.01277840e-01, -5.28127061e-01, -3.32375947e-01,
               2.94492056e-01,  4.07763147e-01],
              [-4.22251570e-01, -5.28127061e-01, -9.58912976e-01,
               2.94492056e-01,  4.07763147e-01],
              [-4.81810463e-01, -2.81906892e-02,  2.82558545e-01,
               2.94492056e-01,  4.07763147e-01],
              [-7.59695726e-01,  4.23862678e-01, -4.48401323e-01,
               2.94492056e-01,  4.07763147e-01],
              [-8.48074137e-02, -5.28127061e-01, -8.89297751e-01,
               2.94492056e-01, -2.45240407e+00],
              [-4.19720739e-01,  1.61136191e-01,  5.05077934e-02,
               2.94492056e-01,  4.07763147e-01],
              [-5.78994380e-01, -2.90602156e-01, -6.34041924e-01,
               2.09071068e+00,  4.07763147e-01],
              [-3.20849601e-01, -2.72456299e-02, -2.59278427e-02,
               2.94492056e-01,  4.07763147e-01],
              [-3.68935393e-01,  5.74442127e-01, -7.03657150e-01,
               2.94492056e-01,  4.07763147e-01],
              [-4.20058183e-01,  4.27327895e-01, -6.22439387e-01,
               -6.03617257e-01,  4.07763147e-01],
              [-5.10155772e-01, -5.28127061e-01, -1.04013074e+00,
               2.94492056e-01,  4.07763147e-01],
```

[-1.11296780e-01, 6.70838176e-01, 2.73027182e-02,
 2.94492056e-01, 4.07763147e-01],
 [-3.96099648e-01, 3.22931063e-02, -1.00325195e-01,
 2.94492056e-01, 4.07763147e-01],
 [-1.52296245e-01, -5.28127061e-01, -8.77695213e-01,
 -2.39983588e+00, 4.07763147e-01],
 [-5.55035845e-01, -1.73729822e-01, -2.59278427e-02,
 2.94492056e-01, 4.07763147e-01],
 [-8.48074137e-02, 6.27050428e-01, 1.01351841e+00,
 2.94492056e-01, 4.07763147e-01],
 [-1.25638157e-01, -5.28127061e-01, 1.08520481e-01,
 2.09071068e+00, 4.07763147e-01],
 [2.31546483e-01, -5.28127061e-01, 5.95827060e-01,
 2.94492056e-01, 4.07763147e-01],
 [1.30917439e+00, -5.28127061e-01, -1.26057895e+00,
 2.94492056e-01, 4.07763147e-01],
 [4.77543272e-01, 6.53197069e-01, 4.44994071e-01,
 2.94492056e-01, 4.07763147e-01],
 [-5.98059975e-01, 2.39261094e-01, -2.04748033e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.51725741e-01, -5.28127061e-01, -2.51158184e-01,
 2.94492056e-01, 4.07763147e-01],
 [-2.85080520e-01, -5.28127061e-01, -3.78786097e-01,
 -2.39983588e+00, 4.07763147e-01],
 [-2.35644951e-01, -5.28127061e-01, -1.26057895e+00,
 -2.39983588e+00, 4.07763147e-01],
 [-3.88507154e-01, -3.07613223e-01, -9.12502826e-01,
 2.94492056e-01, 4.07763147e-01],
 [-5.35295362e-01, 8.85051618e-01, -5.64426699e-01,
 2.94492056e-01, 4.07763147e-01],
 [-4.43341829e-01, 2.31575330e-02, -5.06414011e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.83614214e-01, 3.34712084e-01, -4.48401323e-01,
 2.94492056e-01, 4.07763147e-01],
 [7.58802976e-01, -3.30412750e-03, 8.85890499e-01,
 2.94492056e-01, 4.07763147e-01],
 [1.00111984e-01, -5.28127061e-01, 8.04672736e-01,
 2.94492056e-01, -2.45240407e+00],
 [-3.02796338e-01, 2.81158723e-01, 2.82558545e-01,
 2.94492056e-01, 4.07763147e-01],
 [3.38042071e-02, -5.28127061e-01, -2.39555646e-01,
 2.94492056e-01, 4.07763147e-01],
 [-4.71518416e-01, -1.60498992e-01, -5.99234311e-01,
 2.94492056e-01, 4.07763147e-01],
 [-6.95918780e-01, 6.44251226e-02, 2.12943320e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.14912371e-02, -5.28127061e-01, -1.46735346e-01,

2.94492056e-01, 4.07763147e-01],
 [-5.50649071e-01, 2.66982833e-01, -2.39555646e-01,
 2.94492056e-01, 4.07763147e-01],
 [7.58802976e-01, -5.28127061e-01, 7.37128685e-02,
 2.94492056e-01, 4.07763147e-01],
 [-4.58526816e-01, 1.06952791e-01, -4.48401323e-01,
 2.94492056e-01, 4.07763147e-01],
 [-5.88611539e-01, 7.95460715e-02, -8.66092676e-01,
 2.94492056e-01, 4.07763147e-01],
 [-2.67533424e-01, -5.28127061e-01, -2.85965797e-01,
 2.94492056e-01, 4.07763147e-01],
 [-2.25521627e-01, 5.33174537e-01, 6.07429597e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.94243705e-01, 1.22073740e-01, -5.18016548e-01,
 2.94492056e-01, -2.45240407e+00],
 [-2.62303040e-01, 1.78021970e-02, 4.09764304e-03,
 2.94492056e-01, -2.45240407e+00],
 [-8.48074137e-02, -5.28127061e-01, -5.29619086e-01,
 2.94492056e-01, -2.45240407e+00],
 [-4.61226370e-01, -4.23665787e-02, 4.79801684e-01,
 2.94492056e-01, 4.07763147e-01],
 [7.58802976e-01, -5.28127061e-01, 7.58262586e-01,
 2.94492056e-01, 4.07763147e-01],
 [-6.24718063e-01, 3.96140938e-01, -6.45644462e-01,
 2.94492056e-01, -2.45240407e+00],
 [-3.54762738e-01, -5.28127061e-01, -6.22439387e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.87157377e-01, 4.37723548e-01, 2.70956007e-01,
 2.94492056e-01, 4.07763147e-01],
 [-5.43562744e-01, -5.28127061e-01, -4.13593710e-01,
 2.94492056e-01, 4.07763147e-01],
 [-2.95710011e-01, 9.68216837e-01, 3.17366158e-01,
 2.94492056e-01, 4.07763147e-01],
 [3.85421018e-01, -5.28127061e-01, 1.05992856e+00,
 2.94492056e-01, 4.07763147e-01],
 [-4.58189372e-01, 7.57658343e-02, -3.07099697e-02,
 2.94492056e-01, -2.45240407e+00],
 [1.75024586e-01, -5.28127061e-01, 6.53839748e-01,
 2.94492056e-01, 4.07763147e-01],
 [5.82029659e+00, -5.28127061e-01, 5.23684209e+00,
 -2.39983588e+00, -2.45240407e+00],
 [-3.01277840e-01, 3.93305760e-01, 2.73027182e-02,
 2.94492056e-01, 4.07763147e-01],
 [1.90715740e-01, -5.28127061e-01, -2.59278427e-02,
 2.94492056e-01, -2.45240407e+00],
 [2.76595277e-01, -5.28127061e-01, -1.23530270e-01,
 2.94492056e-01, 4.07763147e-01],

[-4.05379362e-01, -8.70993858e-02, -4.13593710e-01,
 2.94492056e-01, 4.07763147e-01],
 [8.39146643e-02, -5.28127061e-01, 8.53154061e-02,
 2.94492056e-01, 4.07763147e-01],
 [-2.95710011e-01, 1.28059116e-01, -3.32375947e-01,
 2.94492056e-01, 4.07763147e-01],
 [1.88367307e+00, 1.80667417e-01, -7.26862225e-01,
 2.94492056e-01, 4.07763147e-01],
 [-4.26268942e-02, -5.28127061e-01, -6.34041924e-01,
 2.94492056e-01, 4.07763147e-01],
 [8.94824929e-02, -5.28127061e-01, 1.31725556e-01,
 2.94492056e-01, 4.07763147e-01],
 [-4.69493752e-01, -5.28127061e-01, -7.96477450e-01,
 2.50592619e-02, -2.45240407e+00],
 [-4.04873196e-01, -1.18601363e-01, -7.96477450e-01,
 2.94492056e-01, 4.07763147e-01],
 [-1.95489097e-01, -2.96272512e-01, -7.15259687e-01,
 2.94492056e-01, 4.07763147e-01],
 [-7.77210864e-02, 1.28059116e-01, 4.21788996e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.94074983e-01, 7.31952011e-01, 3.63776308e-01,
 -6.03617257e-01, -2.45240407e+00],
 [-5.24328427e-01, -5.28127061e-01, -2.59278427e-02,
 2.94492056e-01, 4.07763147e-01],
 [-3.36203310e-01, 8.63000235e-01, 1.09473618e+00,
 2.94492056e-01, 4.07763147e-01],
 [-9.40871280e-02, -5.28127061e-01, -2.59278427e-02,
 2.94492056e-01, -2.45240407e+00],
 [-3.91881596e-01, -5.28127061e-01, -9.00900289e-01,
 2.94492056e-01, -2.45240407e+00],
 [-3.37890531e-01, -5.28127061e-01, -7.84874913e-01,
 -6.03617257e-01, 4.07763147e-01],
 [-1.97345040e-01, 2.43986390e-01, -4.48401323e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.95424759e-01, -3.82902948e-01, -4.71606398e-01,
 -3.83681078e+00, 4.07763147e-01],
 [1.52415828e-01, -5.28127061e-01, 1.57001806e-02,
 2.94492056e-01, 4.07763147e-01],
 [-5.27196702e-01, -5.28127061e-01, -1.62025762e+00,
 2.94492056e-01, 4.07763147e-01],
 [-2.84574354e-01, -2.90602156e-01, 1.31725556e-01,
 2.94492056e-01, 4.07763147e-01],
 [5.32072461e-02, 1.52315638e-01, 4.10186458e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.51894463e-01, 2.31575330e-02, 4.33391534e-01,
 2.94492056e-01, 4.07763147e-01],
 [-4.68481419e-01, -5.28127061e-01, -4.94811473e-01,

2.94492056e-01, -2.45240407e+00],
 [-5.34789196e-01, -5.05570927e-02, -6.22439387e-01,
 2.94492056e-01, 4.07763147e-01],
 [2.55856138e+00, -5.28127061e-01, -2.59278427e-02,
 2.94492056e-01, 4.07763147e-01],
 [-2.53529492e-01, 1.91327614e+00, 1.64005544e+00,
 2.94492056e-01, 4.07763147e-01],
 [1.68275703e-01, -5.28127061e-01, -5.06414011e-01,
 2.94492056e-01, -2.45240407e+00],
 [-6.46989378e-01, 6.97299837e-01, 6.07429597e-01,
 2.94492056e-01, -2.45240407e+00],
 [1.38749340e-01, -5.28127061e-01, 3.05763620e-01,
 2.94492056e-01, 4.07763147e-01],
 [-2.72595087e-01, 3.12660700e-01, 1.54930632e-01,
 2.94492056e-01, 4.07763147e-01],
 [-5.83381154e-01, -1.81100566e-02, -5.52824161e-01,
 2.94492056e-01, 4.07763147e-01],
 [-5.06612609e-01, -5.28127061e-01, -6.45644462e-01,
 2.94492056e-01, 4.07763147e-01],
 [-3.66067118e-01, 1.01912475e-01, -5.76029236e-01,
 2.94492056e-01, 4.07763147e-01],
 [-6.05146302e-01, 1.06681802e+00, -9.47310439e-01,
 2.94492056e-01, 4.07763147e-01],
 [-4.25626011e-01, 1.28059116e-01, -3.32375947e-01,
 2.94492056e-01, 4.07763147e-01],
 [-2.65846203e-01, -2.76111247e-01, -4.25196248e-01,
 2.94492056e-01, 4.07763147e-01],
 [-4.64432089e-01, 5.21393517e-02, -3.90388635e-01,
 2.94492056e-01, 4.07763147e-01],
 [-2.81706079e-01, -5.28127061e-01, -4.48401323e-01,
 2.94492056e-01, 4.07763147e-01],
 [-8.14699123e-01, 1.14021763e+00, 2.24545857e-01,
 2.94492056e-01, 4.07763147e-01],
 [9.79185968e-02, 8.10706953e-01, 2.10415695e+00,
 2.94492056e-01, 4.07763147e-01],
 [-5.38838526e-01, -1.32777252e-01, -2.74363259e-01,
 2.94492056e-01, -2.45240407e+00],
 [-4.78604744e-01, 8.26457942e-01, -3.20773409e-01,
 2.94492056e-01, 4.07763147e-01],
 [-1.26144323e-01, -5.28127061e-01, -6.22439387e-01,
 2.50592619e-02, -2.45240407e+00],
 [-2.96047455e-01, -2.67408796e-03, -4.48401323e-01,
 2.94492056e-01, 4.07763147e-01],
 [-4.23095180e-01, -5.28127061e-01, -1.02852820e+00,
 2.94492056e-01, 4.07763147e-01],
 [-2.85249242e-01, -5.28127061e-01, -4.25196248e-01,
 2.94492056e-01, 4.07763147e-01],

```

[-3.88507154e-01,  1.81927496e-01, -2.62760721e-01,
 -2.39983588e+00, -2.45240407e+00],
[-3.03471227e-01,  1.01912475e-01, -3.32375947e-01,
 2.94492056e-01,  4.07763147e-01],
[-4.46374703e-04, -1.31202153e-01,  2.47750932e-01,
 2.94492056e-01,  4.07763147e-01],
[-5.23484816e-01, -5.28127061e-01, -8.54490138e-01,
 2.94492056e-01,  4.07763147e-01],
[-2.70064255e-01, -3.30412750e-03, -4.60003860e-01,
 2.94492056e-01,  4.07763147e-01],
[-5.06612609e-01,  6.67687978e-01, -3.32375947e-01,
 2.94492056e-01,  4.07763147e-01],
[ 1.10267266e-02,  1.46645282e-01,  3.05763620e-01,
 2.94492056e-01,  4.07763147e-01],
[-3.01446562e-01, -5.28127061e-01, -1.23737388e+00,
 -2.39983588e+00,  4.07763147e-01],
[-5.05431554e-01, -5.28127061e-01, -1.07493835e+00,
 2.94492056e-01,  4.07763147e-01],
[-4.42160775e-01,  5.24543714e-02, -2.97568334e-01,
 2.09071068e+00,  4.07763147e-01],
[-5.06612609e-01,  5.15093121e-02, -4.60003860e-01,
 2.94492056e-01,  4.07763147e-01],
[ 9.75793245e+00, -5.28127061e-01,  3.96056296e+00,
 -2.39983588e+00,  4.07763147e-01]])

```

```
[89]: X_trained_scaled
```

```

[89]: array([[ 0.08391466,  0.18066742,  1.349992 ,  0.29449206,  0.40776315],
 [-0.4293379 ,  0.38543027, -0.20474803,  0.29449206,  0.40776315],
 [ 0.12609518,  0.00583145,  0.71185244,  0.29449206,  0.40776315],
 ...,
 [-0.38226444, -0.52812706, -1.37660433,  0.29449206,  0.40776315],
 [ 0.76673291, -0.52812706, -0.02592784, -1.50172657,  0.40776315],
 [ 1.37463856, -0.52812706, -0.02592784,  0.29449206,  0.40776315]],
 shape=(491, 5))

```

```

[90]: from sklearn.linear_model import LogisticRegression # import the logistic
      ↪ regression from the sklearn library
      model = LogisticRegression() # creating the model
      model.fit(X_trained_scaled,y_train) # training the model
      predictions = model.predict(X_test_scaled) # make predictions

```

```
[91]: predictions
```

```

[91]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1,

```

```
1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0,
1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1,
1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

```
[92]: from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, predictions)
```

```
[93]: print(f"The accuracy Score of Logistic Regression is : {accuracy:.2}")
```

The accuracy Score of Logistic Regression is : 0.79

```
[94]: from sklearn.metrics import
      ↪ confusion_matrix, ConfusionMatrixDisplay, r2_score, classification_report
cm = confusion_matrix(y_test, predictions)

print(cm)
```

```
[[18 25]
 [ 1 79]]
```

```
[95]: r2_score(y_test, predictions)
```

```
[95]: 0.0703488372093023
```

```
[96]: classification_report(y_test, predictions)
```

```
[96]: '
      precision    recall  f1-score   support\n\n
0.95      0.42      0.58      0.43      1
0.85      0.70      0.72      0.83      123\n\n
accuracy          0.79
weighted avg      0.83
macro avg         0.79
0.95      0.42      0.58      0.43      1
0.85      0.70      0.72      0.83      123\n\n
accuracy          0.79
weighted avg      0.83
macro avg         0.79'
```

```
[97]: from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier (random_state =42,n_estimators = 100,max_depth =
      ↪ 5,min_samples_split = 2)
rf.fit(X_trained_scaled,y_train)
new_pred = rf.predict(X_test_scaled)
new_pred
```

```
[97]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1,
1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,
1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1,
1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

```
[98]: acc1 = accuracy_score(y_test, new_pred)
```

```
[99]: print(f"The accuracy Score of Random Forest Classifier is : {acc1:.5f}")
```

The accuracy Score of Random Forest Classifier is : 0.78049

```
[100]: from sklearn.naive_bayes import GaussianNB
gaussian = GaussianNB()
gaussian.fit(X_trained_scaled,y_train)

new_pred_gaussian = gaussian.predict(X_test)
new_pred_gaussian
```

```
C:\Users\User\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2742:
UserWarning: X has feature names, but GaussianNB was fitted without feature
names
```

```
warnings.warn(
```

```
[100]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
[101]: accuracy_score1 = accuracy_score(y_test,new_pred_gaussian)
```

```
[102]: print(f"The accuracy Score for Naive Bayes is : {accuracy_score1:.5f}")
```

The accuracy Score for Naive Bayes is : 0.34959

```
[103]: from sklearn.svm import SVC
svm = SVC(gamma = "scale", C = 0.1, random_state = 42)
svm.fit(X_trained_scaled, y_train)
svm_predict = svm.predict(X_test_scaled)
svm_predict
```

```
[103]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
            1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1,
            1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1,
            1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,
            1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1,
            1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

```
[104]: svm_accuracy2 = accuracy_score(y_test,svm_predict)
```

```
[105]: print(f"The accuracy Score of Support Vector Machine is : {svm_accuracy2:.5f}")
```

The accuracy Score of Support Vector Machine is : 0.79675

```
[106]: from sklearn.linear_model import LinearRegression
log = LinearRegression()
log.fit(X_train_scaled, y_train)
```

```
log_pred = log.predict(X_test_scaled)
log_pred
```

```
[106]: array([0.79591744, 0.81642079, 0.79836502, 0.82865846, 0.82887734,
0.80235138, 0.81852066, 0.82967962, 0.79419932, 0.79507198,
0.11516235, 0.79317468, 0.81777628, 0.79957328, 0.79536351,
0.79781697, 0.83115721, 0.77969203, 0.79931704, 0.82782177,
0.80362889, 0.76337972, 0.81083045, 0.8018368 , 0.83428283,
0.77251047, 0.79566714, 0.81710256, 0.81903445, 0.83464518,
0.82288875, 0.78458073, 0.80678294, 0.79731722, 0.78229832,
0.08505295, 0.78577025, 0.81673109, 0.81339905, 0.79302497,
0.81511369, 0.79551535, 0.81086584, 0.80349868, 0.81169919,
0.81768341, 0.77317372, 0.09115884, 0.08466153, 0.10878614,
0.79107657, 0.79873048, 0.08612097, 0.82368576, 0.78178513,
0.82006464, 0.76660035, 0.79354326, 0.08379787, 0.80083268,
0.00371977, 0.7872668 , 0.09973847, 0.81456984, 0.80809796,
0.81095038, 0.80080197, 0.80543765, 0.82375721, 0.81012525,
0.11365211, 0.81573583, 0.81900293, 0.78733876, 0.05896009,
0.81318402, 0.75567766, 0.09986094, 0.11550009, 0.82645742,
0.7996866 , 0.81664544, 0.81215503, 0.84144874, 0.80387321,
0.78683324, 0.79008321, 0.10833409, 0.81086932, 0.81185826,
0.71761868, 0.10826594, 0.05578571, 0.80701881, 0.78716926,
0.80878008, 0.82416243, 0.80585752, 0.78649086, 0.80085784,
0.81334675, 0.8039527 , 0.82056908, 0.76382518, 0.73900832,
0.09378234, 0.78181895, 0.11041919, 0.80638861, 0.83091409,
0.82015924, 0.08471336, 0.80151124, 0.79739056, 0.827872 ,
0.80660013, 0.78632329, 0.78885563, 0.83426211, 0.83177223,
0.80249046, 0.80522195, 0.73778973])
```